CAMPUS DE EXCELENCIA INTERNACIONAL

Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

Metodologías de Desarrollo Web

Máster de Ingeniería Web
E.T.S. de Ingeniería en Sistemas Informáticos
Universidad Politécnica de Madrid
Curso 2025-2026



Jennifer Pérez y Agustín Yagüe
Universidad Politécnica de Madrid
E.T.S. de Ingeniería en Sistemas Informáticos
Departamento de Sistemas Informáticos

CAMPUS DE EXCELENCIA INTERNACIONAL

Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

- **CONTENIDOS**
  - Metodologías Ágiles: Valores y Principios
  - Scrum
    - Proceso
    - Historias de Usuario y Equipo Scrum
    - Técnicas de Adopción
      - Gestión: Estimación, Organización y Planificación
      - Desarrollo: Buenas Prácticas: Patrones Software
  - Conclusiones
  - Bibliografía



Metodologías Ágiles: Valores y Principios



Manifiesto Ágil

- Toda metodología ágil así como su aplicación debe cumplirlo
- Consta de:
  - **4 valores**
  - **12 principios**
- Propuesto por:
  - Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, Dave Thomas



Manifiesto Ágil

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan



Manifiesto Ágil

Individuals and interactions over processes and tools

- **¿Por qué?**
  - Si se gestionan personas como componentes y no se trata a personas como individuos
    - Se reduce la motivación
    - Se baja la productividad
    - La gente buena se va
  - Los individuos e interacciones son esenciales para los equipos de alto rendimiento



Manifiesto Ágil



Individuals and interactions over processes and tools

– ¿Cómo lograrlo?

- Aumentando la frecuencia de los comentarios y la comunicación
  - Ciclos de inspección y adaptación frecuentes.
- Comportamiento de los miembros del equipo:
  - Respeto por el valor de cada persona
  - Veracidad en cada comunicación
  - Transparencia de todos los datos, acciones y decisiones
  - Confianza en que cada persona respaldará al equipo
  - Compromiso con el equipo y los objetivos del equipo



La gente lo evita → *Facilitar el Conflicto Positivo*

Manifiesto Ágil



Individuals and interactions over processes and tools

– Conflicto Positivo

- **En qué consiste:**
  - Generar una lista de impedimentos o problemas en la organización
  - Enfrentarse a ellos directamente
  - Eliminarlos sistemáticamente en orden de prioridad.
- **Mejoras:**
  - Crea un comportamiento productivo
  - Mejora del proceso
  - Incorporación de la Innovación
  - Alinear al equipo hacia un objetivo común
  - Compromiso de trabajar juntos → Mejora personal y del equipo





Manifiesto Ágil

Working software over comprehensive documentation

- Si el software no funciona los documentos no valen de nada
- Documentación
  - Interna: sólo la necesaria.
  - Externa: toda la que pida el cliente.
- Definición de **El Software Funciona = Terminado**





Manifiesto Ágil

Customer collaboration over contract negotiation

- Retroalimentación
  - **Cliente** proporcionar comentarios sobre el software que funciona a intervalos regulares
  - Un representante del cliente (***product owner***) trabaja mano a mano con el equipo de desarrollo
- Mayor Tasa de Éxito

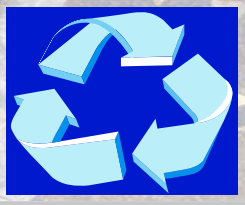


Manifiesto Ágil



Responding to change over following a plan

- Más del 60% de los requisitos de producto o de proyecto cambian durante el desarrollo de software
- Los clientes nunca saben lo que desean hasta que ven el software que funciona
- Si el proyecto no es capaz de adaptarse a los cambios fracasa
- Desarrollo Adaptativo:
  - Satisfacción requisitos del cliente
  - Valor comercial



Manifiesto Ágil



1. La prioridad principal es satisfacer al cliente mediante tempranas y continuas entregas de software utilizable
2. Dar la bienvenida a los cambios. Los procesos ágiles aplican los cambios para que el cliente sea competitivo
3. Entregar el software desarrollado frecuentemente con el menor intervalo de tiempo posible entre una entrega y la siguiente
4. La gente de negocios y los desarrolladores trabajan juntos a través de un proyecto
5. Construir proyecto empujados por motivaciones personales. Dar el entorno que necesitan las personas y confiar en ellos.
6. El diálogo cara a cara es el método más eficiente y efectivo para comunicar información dentro de un equipo de desarrollo

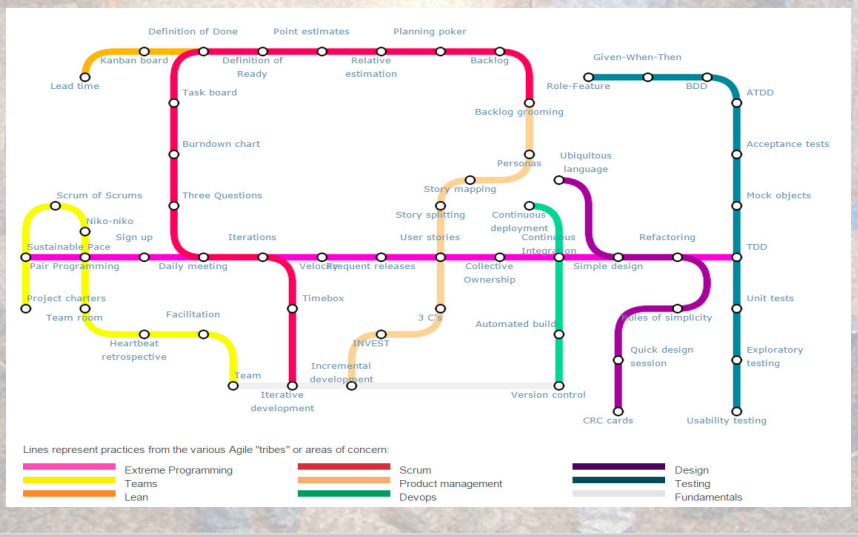


Manifiesto Ágil

Principios

7	•Desarrollar software es la primera medida de progreso.
8	•Los procesos ágiles promueven un desarrollo llevadero. Los patrocinadores, desarrolladores y usuarios son capaces de mantener una paz constante
9	•La atención continua a la calidad técnica y al buen diseño incrementa la agilidad
10	•La simplicidad es esencial
11	•Las mejores arquitecturas, requisitos y diseños surgen de la propia organización del equipo
12	•En intervalos regulares, el equipo reflexiona en cómo llegar a ser más efectivo, sincronizar y ajustar su comportamiento

Prácticas Ágiles



Prácticas Ágiles

- Se organizan alrededor de 9 categorías principales.
  - Extreme Programming
  - Teams
  - Lean
  - Scrum
  - Product Management
  - DevOps
  - Design
  - Testing
  - Fundamentals
- Una práctica puede pertenecer a más de una categoría

Prácticas Ágiles

- **Fundamentals (línea gris)**
  - **Team**

    A “team” in the Agile sense is a small group of people, assigned to the same project or effort, nearly all of them on a full-time basis. A small minority of team members may be part-time contributors, or may have competing responsibilities.
  - **Iterative development**

    Agile projects are iterative insofar as they intentionally allow for “repeating” software development activities, and for potentially “revisiting” the same work product, and in being most often structured around a series of iterations of fixed calendar length. .
  - **Incremental development**

    his means that each successive version of the product is usable, and each builds upon the previous version by adding user-visible functionality. These are called “vertical” increments (that is, difference between successive product versions),
  - **Version control**

    is not merely “good practice” but an enabler of a number of Agile practices, such as continuous integration

Prácticas Ágiles

- Teams (Línea amarilla)
  - **Team** (link con Fundamentals)
  - **Facilitation**

    A facilitator is a person who chooses or is given the explicit role of conducting a meeting. This role usually entails that the facilitator will take little part in the discussions on the meeting's topic, but will focus primarily on creating the conditions for effective group processes, in the pursuit of **the objectives for which the meeting was convened.**
  - **Heartbit retrospective**

    The team meets regularly, usually adhering to the rhythm of its iterations, to explicitly reflect on the most significant events to have occurred since the previous such meeting, and take decisions aiming at remediation or improvement.
  - **Team room**

    The team (ideally the whole team, including the product owner or domain expert) has the use of a dedicated space for the duration of the project, set apart from other groups' activities. It is furnished with the various amenities that the team may need.

Prácticas Ágiles

- Teams (Línea amarilla)
  - **Pair programming** (Link con Extreme Programming)

    Pair programming consists of two programmers sharing a single workstation (one screen, keyboard and mouse among the pair). The programmer at the keyboard is usually called the "driver", the other, also actively involved in the programming task but focusing more on overall direction is the "navigator"; it is expected that the programmers swap roles every few minutes or so.
  - **Niko-Niko**

    The team installs a calendar on one of the room's walls. The format of the calendar allows each team member to record, at the end of every workday, a graphic evaluation of their mood during that day. This can be either a hand-drawn "emoticon", or a colored sticker, following a simple color code, for instance: blue for a bad day, red for neutral, yellow for a good day. Over time, the niko-niko calendar reveals patterns of change in the moods of the team, or of individual members.

Prácticas Ágiles

- Teams (Línea amarilla)
  - **Scrum of Scrums**
    A technique to scale Scrum up to large groups (over a dozen people), consisting of dividing the groups into Agile teams of 5-10.
  - **Sustainable Pace** (Link con Extreme Programming)
    The team aims for a work pace that they would be able to sustain indefinitely. This entails a firm refusal of what is often considered a "necessary evil" in the software industry – long work hours, overtime, or even working nights or weekends.
  - **Project charter**
    The team develops and maintains a high-level summary of the project's key success factors, synthetic enough that it can be displayed on one wall of the team room as a flipchart-sized sheet of paper.

Prácticas Ágiles

- **Scrum (Línea roja)**
  - **Iterative development** (link Fundamentals)
  - **Timebox**
    A timebox is a previously agreed period of time during which a person or a team works steadily towards completion of some goal. Rather than allow work to continue until the goal is reached, and evaluating the time taken, the timebox approach consists of stopping work when the time limit is reached and evaluating what was accomplished.
  - **Iterations** (link con Extreme programming)
    An iteration, in the context of an Agile project, is a timebox during which development takes place, the duration of which:
    - may vary from project to project, usually between 1 and 4 weeks
    - is in most cases fixed for the duration of a given project

Prácticas Ágiles

- **Scrum (Línea roja)**
  - **Daily meeting** (link con Extreme programming)

    Each day at the same time, the team meets so as to bring everyone up to date on the information that is vital for coordination: each team members briefly describes any "completed" contributions and any obstacles that stand in their way. Usually, Scrum's Three Questions are used to structure discussion. The meeting is normally held in front of the task board. It is timeboxed.
  - **Three questions**

    The daily meeting is structured around some variant of the following three questions:
    - What have you completed since the last meeting?
    - What do you plan to complete by the next meeting?
    - What is getting in your way?
  - **Task board**

    In its most basic form, a task board can be drawn on a whiteboard or even a section of wall. Using electrical tape or a dry erase pen, the board is divided into three columns labeled "To Do", "In Progress" and "Done". Sticky notes or index cards, one for each task the team is working on, are placed in the columns reflecting the current status of the tasks.

CAMPUS DE EXCELENCIA INTERNACIONAL
Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

Prácticas Ágiles

- **Scrum (Línea roja)**
  - **Burndown chart**

    The team displays, somewhere on a wall of the project room, a large graph relating the quantity of work remaining (on the vertical axis) and the time elapsed since the start of the project (on the horizontal, showing future as well as past). This constitutes an "information radiator", provided it is updated regularly.
  - **Definition of done** (Link con Lean)

    The team agrees on, and displays prominently somewhere in the team room, a list of criteria which must be met before a product increment "often a user story" is considered "done".
  - **Definition of ready**

    By analogy with the "Definition of Done", the team makes explicit and visible the criteria (generally based on the INVEST matrix) that a user story must meet prior to being accepted into the upcoming iteration.

Prácticas Ágiles

- **Scrum (Línea roja)**
  - **Point estimates**
    Agile teams generally prefer to express estimates in units other than the time-honored “man-day” or “man-hour.” Possibly the most widespread unit is “story points.”
  - **Relative estimation**
    Relative estimation is one of the several distinct flavors of estimation used in Agile teams, and consists of estimating tasks or user stories, not separately and in absolute units of time, but by comparison or by grouping of items of equivalent difficulty.
  - **Planning poker**
    A playful approach to estimation, used by many Agile teams. The team meets in presence of the customer or Product Owner. Around the table, each team member holds a set of playing cards, bearing numerical values appropriate for points estimation of a user story.

Prácticas Ágiles

- **Scrum (Línea roja)**
  - **Backlog**
    A product backlog is a list of the new features, changes to existing features, bug fixes, infrastructure changes or other activities that a team may deliver in order to achieve a specific outcome. The product backlog is the single authoritative source for things that a team works on.
  - **Backlog refinement** (link con Product management)
    Backlog refinement (formerly known as backlog grooming) is when the product owner and some, or all, of the rest of the team review items on the backlog to ensure the backlog contains the appropriate items, that they are prioritized, and that the items at the top of the backlog are ready for delivery. This activity occurs on a regular basis and may be an officially scheduled meeting or an ongoing activity.

Prácticas Ágiles

- Product Management (Línea marrón claro)
  - **Backlog refinement** (link con Scrum)
  - **Personas**

    When the project calls for it – for instance when user experience is a major factor in project outcomes – the team crafts detailed, synthetic biographies of fictitious users of the future product: these are called “personas”.
  - **Story mapping**

    Story mapping consists of ordering user stories along two independent dimensions. The “map” arranges user activities along the horizontal axis in rough order of priority (or “the order in which you would describe activities to explain the behaviour of the system”). Down the vertical axis, it represents increasing sophistication of the implementation.
  - **Story splitting**

    Before a user story is ready to be scheduled for implementation in an upcoming iteration, it should be “small enough,” the usual rule of thumb being “a story that can be completed within the iteration”. However, many user stories start out larger than that. “Splitting” consists of breaking up one user story into smaller ones, while preserving the property that each user story separately has measurable business value.

Prácticas Ágiles

- Product Management (Línea marrón claro)
  - **User stories** (link con Extreme Programming)

    In consultation with the customer or product owner, the team divides up the work to be done into functional increments called “user stories.” Each user story is expected to yield, once implemented, a contribution to the value of the overall product, irrespective of the order of implementation; these and other assumptions as to the nature of user stories are captured by the INVEST formula.
  - **3 Cs**

    “Card, Conversation, Confirmation”; this formula (from Ron Jeffries) captures the components of a User Story.
  - **INVEST**

    The acronym INVEST helps to remember a widely accepted set of criteria, or checklist, to assess the quality of a user story. If the story fails to meet one of these criteria, the team may want to reword it, or even consider a rewrite
  - **Incremental development** (lik con fundamentals)

Prácticas Ágiles

- **Lean (Línea naranja)**
  - **Lead time**
    "Lead time" is a term borrowed from the manufacturing method known as Lean or Toyota Production System, where it is defined as the time elapsed between a customer placing an order and receiving the product ordered. Translated to the software domain, lead time can be described more abstractly as the time elapsed between the identification of a requirement and its fulfilment.
  - **Kanban board**
    The term "kanban" is Japanese (看板), with the sense of a sign, poster or billboard, and derived from roots which literally translate as "visual board". Its meaning within the Agile context is borrowed from the Toyota Production System, where it designates a system to control the inventory levels of various parts.
  - **Definition of done** (link con Scrum)

Prácticas Ágiles

- **Extreme programming (Línea rosa)**
  - **Sustainable pace** (link con Teams)
  - **Pair programming** (link con Teams)
  - **Sign up for tasks**
    Members of an Agile development team normally choose which tasks to work on, rather than being assigned work by a manager. Their choice may be negotiated in discussion with other team members.
  - **Daily meeting** (link con Scrum)
  - **Velocity**
    At the end of each iteration, the team adds up effort estimates associated with user stories that were completed during that iteration. This total is called velocity. Knowing velocity, the team can compute (or revise) an estimate of how long the project will take to complete, based on the estimates associated with remaining user stories and assuming that velocity over the remaining iterations will remain approximately the same.

Prácticas Ágiles

- **Extreme programming (Línea rosa)**
  - **Frequent delivery**

    An Agile team frequently releases its product into the hands of end users, listening to feedback, whether critical or appreciative. Precisely how frequent is desirable varies according to the technical and business aspects of the context, but in general one release every four to six iterations would be considered a maximum.
  - **User Stories** (link con Produt management)
  - **Collective ownership**

    Teams typically adopt conventions governing who is allowed to modify some source code that was originally written by another, often referred to as "ownership". These conventions can be written and explicit, merely oral, or entirely implicit.
  - **Continuous integration** (link con DevOps)

    Teams practicing continuous integration seek two objectives:
    - minimize the duration and effort required by each integration episode
    - be able to deliver a product version suitable for release at any moment

Prácticas Ágiles

- **Extreme programming (Línea rosa)**
  - **Simple design**

    A team adopting the "simple design" practice bases its software design strategy on the following principles: includes refactoring and heuristics such as YAGNI, code simplicity, "design patterns", plugin-based architectures, deferred until the "last responsible moment"
  - **Refactoring**

    Refactoring consists of improving the internal structure of an existing program's source code, while preserving its external behaviour.
  - **TDD** (link con testing)

    "Test-driven development" refers to a style of programming in which three activities are tightly interwoven: coding, testing (in the form of writing unit tests) and design (in the form of refactoring).

Prácticas Ágiles

- **Design (Línea morada)**
  - **CRC cards**

    CRC cards (for Class, Responsibilities, Collaborators) are an activity bridging the worlds of role-playing games and object-oriented design. With the intent of rapidly sketching several different ideas for the design of some feature of an object-oriented systems, two or more team members write down on index cards the names of the most salient classes involved in the feature.
  - **Quick desing session**

    When a team favours “simple design,” developers usually handle local design decisions moment-to-moment, but are on the alert for design choices that may have far-reaching consequences. When such a choice arises, two or more developers meet for a quick design session at the whiteboard, possibly using design aids such as CRC cards.

Prácticas Ágiles

- **Design (Línea morada)**
  - **Rules of simplicity**

    A set of criteria, in priority order, proposed by Kent Beck to judge whether some source code is “simple enough”: the code is verified by automated tests, and all such tests pass, the code contains no duplication, the code expresses separately each distinct idea or responsibility, the code is composed of the minimum number of components (classes, methods, lines) compatible with the first three criteria
  - **Refactoring** (link con Extreme programming)
  - **Simple design** (link con Extreme programming)
  - **Ubiquitous language**

    A design approach described in Eric Evans’ “Domain Driven Design” (2003), which consists notably of striving to use the vocabulary of a given business domain, not only in discussions about the requirements for a software product, but in discussions of design as well and all the way into “the product’s source code itself”.

Prácticas Ágiles

- **Testing (Línea turquesa)**
  - **Role-Feature**
    Identify main elements described in user stories to start defining tests
  - **Given-When-Then**
    The Given-When-Then formula is a template intended to guide the writing of acceptance tests for a User Story.
  - **BDD**
    Behaviour Driven Development (BDD) is a synthesis and refinement of practices stemming from Test Driven Development (TDD) and Acceptance Test Driven Development (ATDD).
  - **ATDD**
    Analogous to test-driven development, Acceptance Test Driven Development (ATDD) involves team members with different perspectives (customer, development, testing) collaborating to write acceptance tests in advance of implementing the corresponding functionality.

Prácticas Ágiles

- **Testing (Línea turquesa)**
  - **Acceptance tests**
    An acceptance test is a formal description of the behavior of a software product, generally expressed as an example or a usage scenario. A number of different notations and approaches have been proposed for such examples or scenarios.
  - **Mock objects**
    A technique commonly used in the context of crafting automated unit tests. It consists of instantiating a test-specific version of a software component (typically a class), which insteads of the normal behaviors provides precomputed results, and often also checks that it's invoked as expected by the objects being tested
  - **TDD** (link con Extreme programming)
  - **Unit tests**
    A unit test, as Agile teams understand the term, is a short program fragment written and maintained by the developers on the product team, which exercises some narrow part of the product's source code and checks the results.

Prácticas Ágiles

- **Testing (Línea azul oscuro)**
  - **Exploratory testing**

    Exploratory testing is, more than strictly speaking a “practice,” a style or approach to testing software which is often contrasted to “scripted testing,” and characterized by the following aspects among others: tester’s autonomy, performing various test-related activities in an interleaved manner
  - **Usability testing**

    Usability testing is a long-established, empirical and exploratory technique to answer questions such as “how would an end user respond to our software under realistic conditions?” It consists of observing a representative end user interacting with the product, given a goal to reach but no specific instructions for using the product.

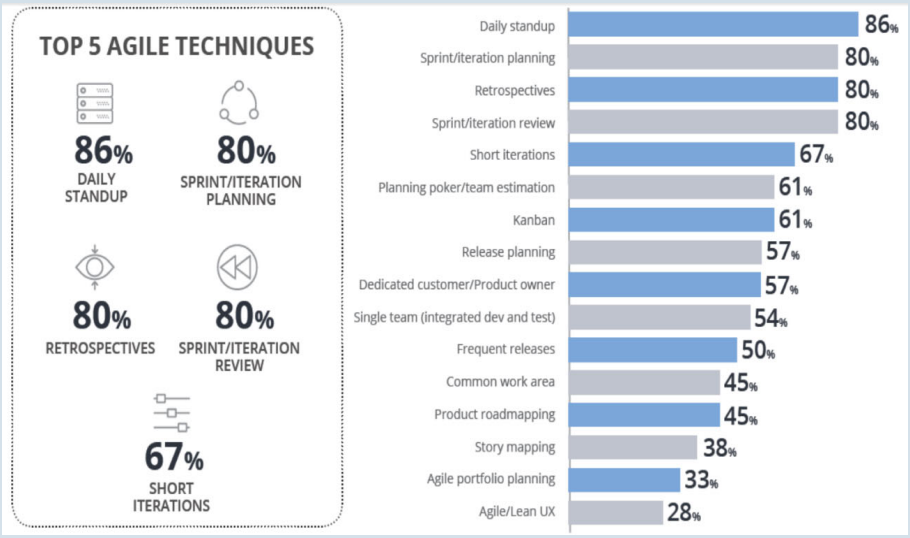
Prácticas Ágiles

- **DevOps (Línea verde oscuro)**
  - **Version control** (link con Fundamentals)
  - **Automated build**

    In the context of software development, build refers to the process that converts files and other assets under the developers’ responsibility into a software product in its final or consumable form. The build may include: compiling source files, packaging compiled, producing installers and creating or updating of database schema or data.
  - **Continuous Integration** (link con Extreme programming)
  - **Continuous Deployment**

    Continuous deployment can be thought of as an extension of continuous integration, aiming at minimizing lead time, the time elapsed between development writing one new line of code and this new code being used by live users, in production.

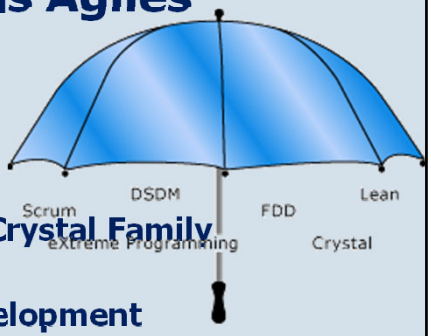
Técnicas Ágiles

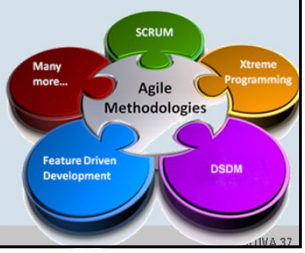


13th annual State of Agile survey report, *VersionOne, Inc, 2019*

Metodologías Ágiles

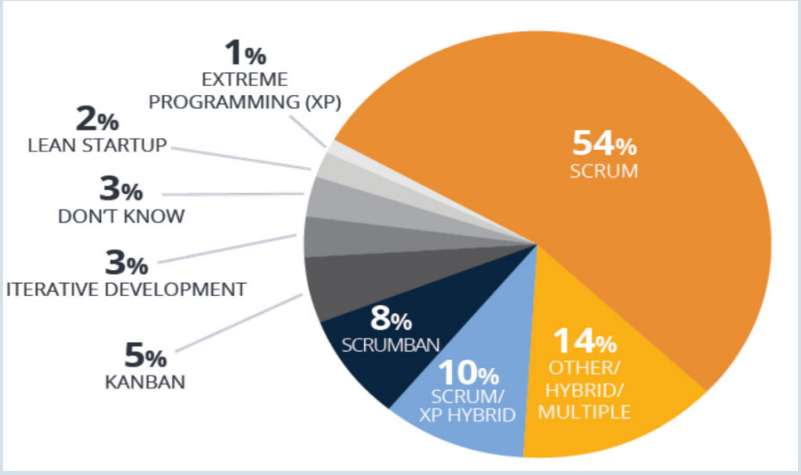
- **Scrum**
- **Extreme Programming (XP)**
- **Lean Software Development**
- **Kanban**
- **Crystal Methods (Cockburn's Crystal Family Methodologies)**
- **DSDM (Dynamic Systems Development Method)**
- **Adaptative Software Development**
- **....**





Metodologías Ágiles

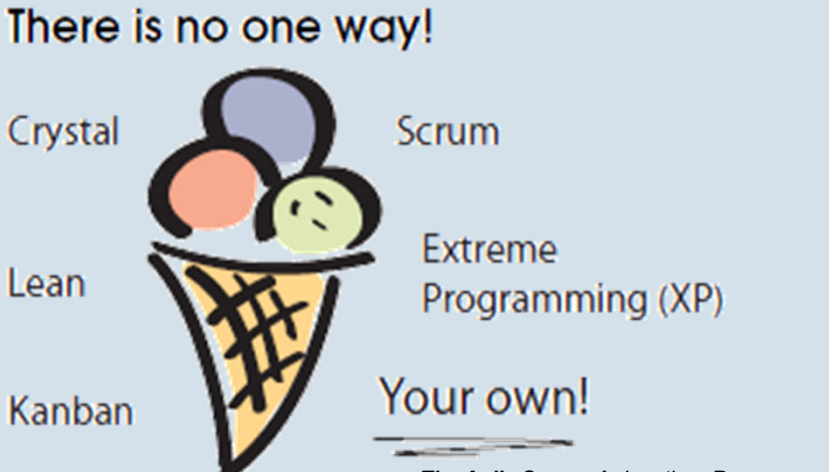
- **Grado de adopción**



***13th annual State of Agile survey report** , Inc, 2019*

Metodologías Ágiles

- **Tu Propia Metodología**



***The Agile Samurai**, Jonathan Rasmusson*

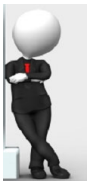
Metodologías Ágiles

- Extreme Programming (XP)

- **Filosofía y Prácticas**

Historias de Usuario





Cliente in situ

Espacios de Trabajo Abiertos



Equipos 5-10

Propiedad Colectiva Programación en Parejas

Refactorización

$$ab + ac + ad = a(b + c + d)$$

Diseño y Arquitectura Incremental

Diseño Sencillos

El Juego de la Planificación



Metodologías Ágiles

- Extreme Programming (XP)

- **Filosofía y Prácticas**

Pequeñas entregas y versiones



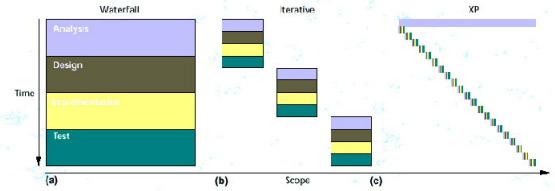
Semanas de 40 horas



DESARROLLO

Estándares de programación

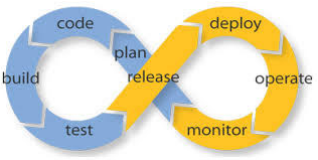
Iteraciones cortas



PRUEBAS

Pruebas Unitarias

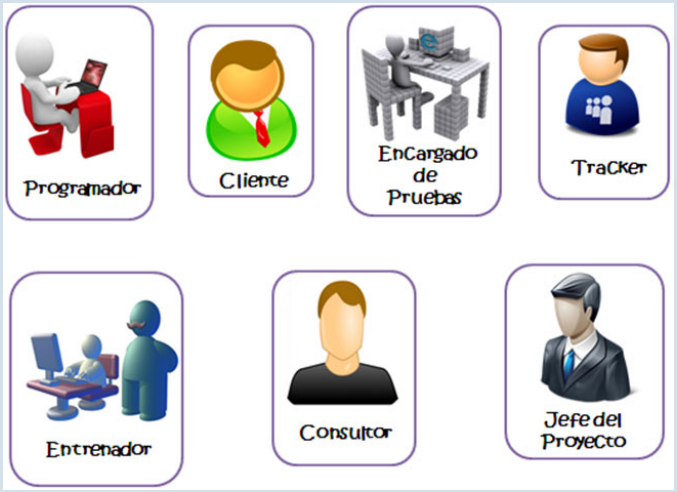
Integración Continua



Metodologías Ágiles

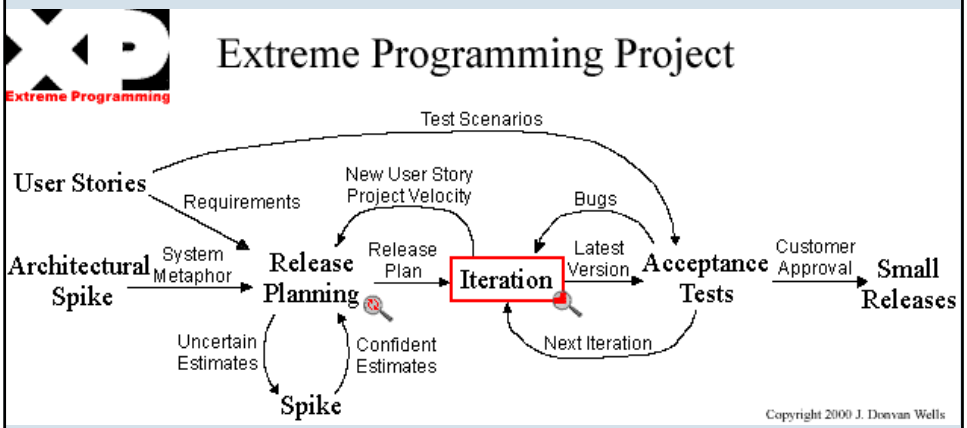
- Extreme Programming (XP)

- **Roles**



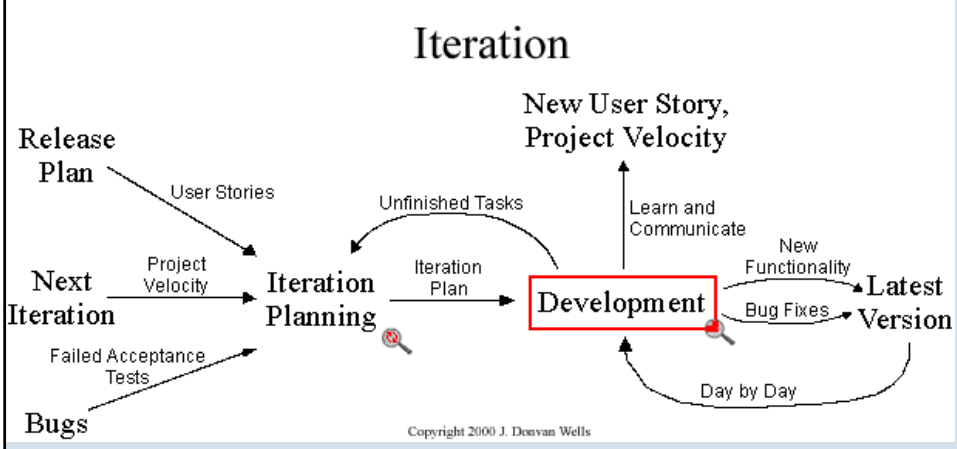
Metodologías Ágiles

- Extreme Programming (XP)



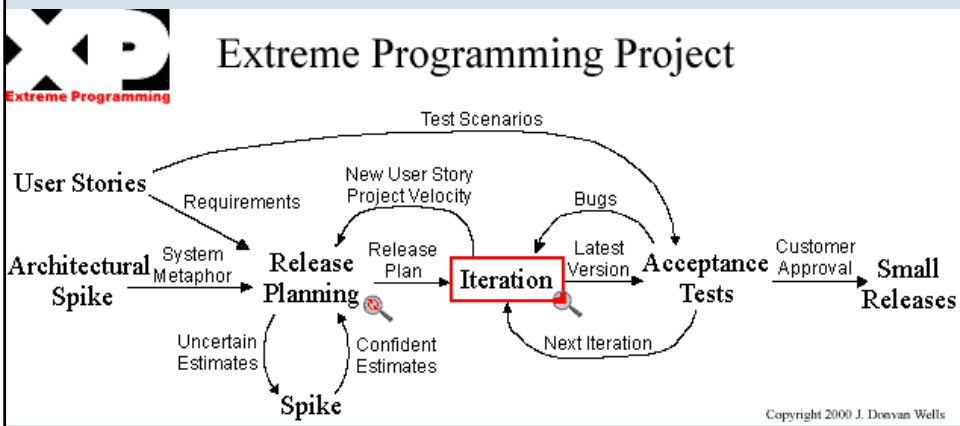
***Extreme Programming Project**, 2013*

Metodologías Ágiles
- Extreme Programming (XP)



Extreme Programming Project, 2013

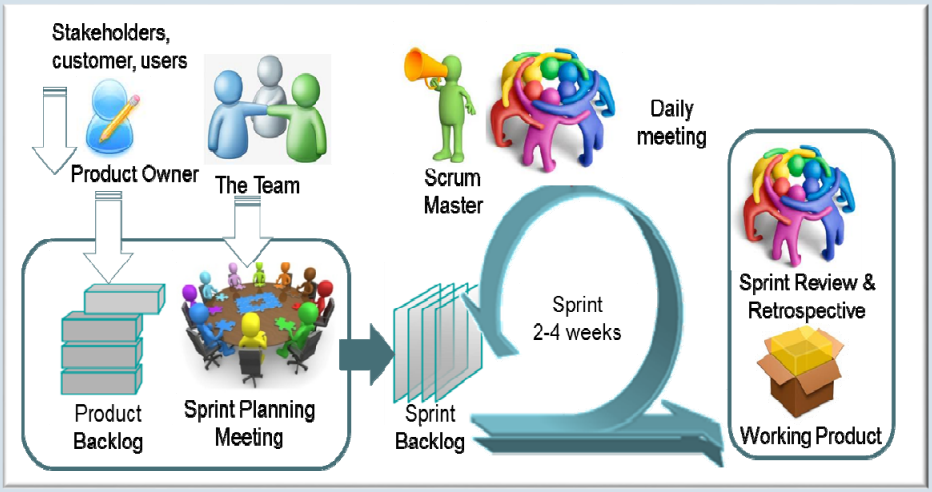
Metodologías Ágiles
- Extreme Programming (XP)



Extreme Programming Project, 2013

Metodologías Ágiles

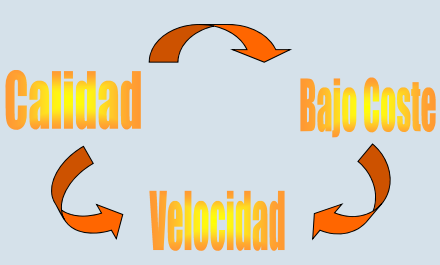
- Scrum

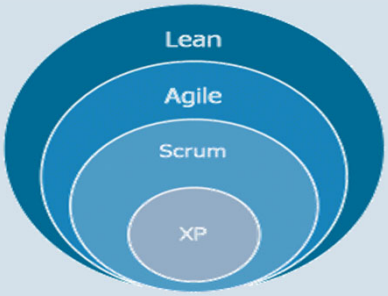


Metodologías Ágiles

- Lean Software Development

- Surge del sistema de producción de Toyota
- Basado en un conjunto extenso de principios económicos y matemáticos que describen el flujo de información del product dentro de la empresa/compañía
- Proporciona un amplio framework económico y de desarrollo





Metodologías Ágiles

- Lean Software Development

Principios



1. Eliminar el gasto



2. Incrementar el aprendizaje



3. Postergar las decisiones



4. Entregar lo antes posible



5. Fortalecer el equipo



6. Incorporar integración



7. Ver todo

Metodologías Ágiles

- Lean Software Development

La Casa Lean

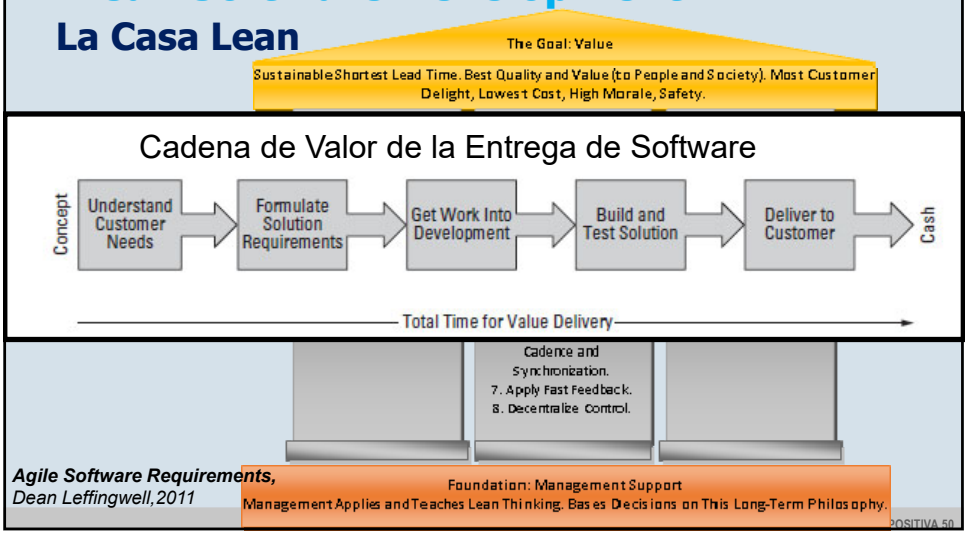


Agile Software Requirements,
Dean Leffingwell, 2011

Metodologías Ágiles

- Lean Software Development

La Casa Lean



Cadena de Valor de la Entrega de Software

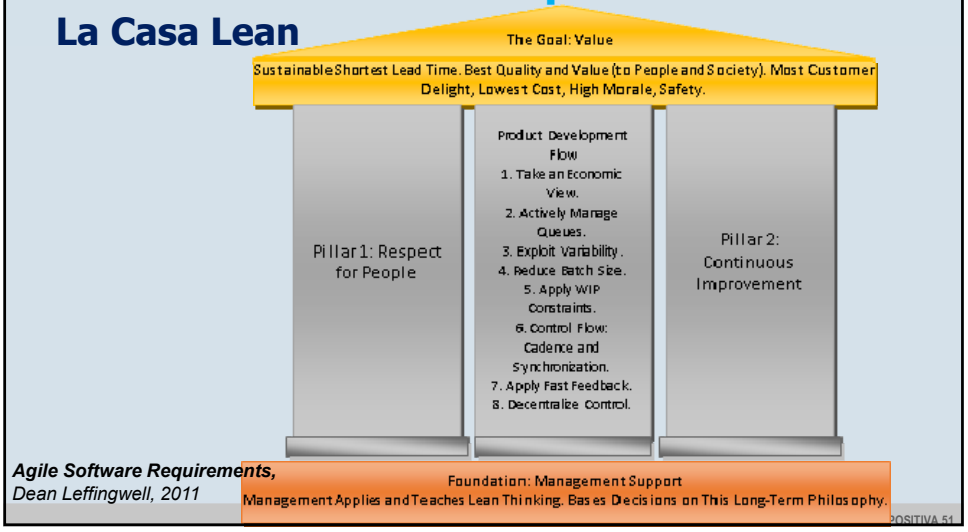
Agile Software Requirements,
Dean Leffingwell,2011

CAMPUS DE EXCELENCIA INTERNACIONAL
Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

Metodologías Ágiles

- Lean Software Development

La Casa Lean



Agile Software Requirements,
Dean Leffingwell, 2011

Metodologías Ágiles

- Kanban

- **En japonés significa *"Señal, Signo"***
- **Forma de planificar y gestionar el trabajo software**
- **Se basa en la idea :**
  - *El trabajo en curso **(Work InProgress, WIP)** debería limitarse, y sólo deberíamos empezar con algo nuevo cuando un bloque de trabajo anterior haya sido entregado o ha pasado a otra función posterior de la cadena*

Metodologías Ágiles

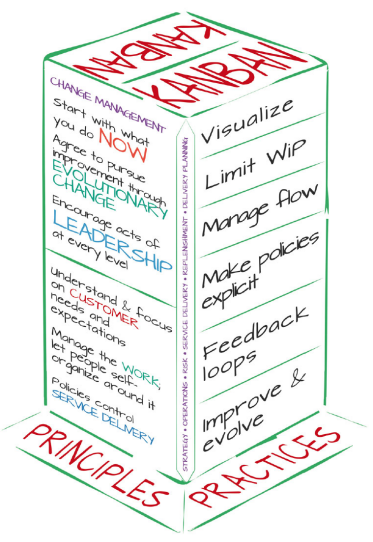
- Kanban

- **El Kanban** (señal) implica que se genera una **señal visual** para indicar que hay n**uevos** bloques de **trabajo** que pueden ser comenzados porque el trabajo en curso actual no alcanza el **máximo acordado**
- **Características**
  - Visualizar unidades de valor (bloques de trabajo) y el flujo de trabajo
  - Gestión y abordar las unidades de valor limitando el trabajo en curso para que sea sostenible y detectando los cuellos de botella
  - Medir y Optimizar el flujo de trabajo: tiempo medio acabar una unidad de valor
  - Mecanismo soporte dónde añadir y quitar trabajo
  - No establece Roles

Metodologías Ágiles

- Kanban

Principios y Roles



Metodologías Ágiles

- Kanban

Principios de la gestión del cambio



- Empieza por lo que haces ahora
  - Comprender los procesos actuales
  - Respetar los roles existentes
- Aceptar buscar la mejora mediante el cambio evolutivo
- Fomenta actos de liderazgo en todos los niveles.

Objetivos
- Minimizar la resistencia al cambio
- Los procesos actuales contienen la sabiduría de los trabajadores

Metodologías Ágiles

- Kanban

Principios de prestación de servicios



- Comprende y céntrate en las necesidades y expectativas de tus clientes
- Gestionar el trabajo; Deja que la gente se autoorganice en torno a ello
- Evolucionar políticas para mejorar los resultados tanto para clientes como para el negocio.

Metodologías Ágiles

- Kanban Roles

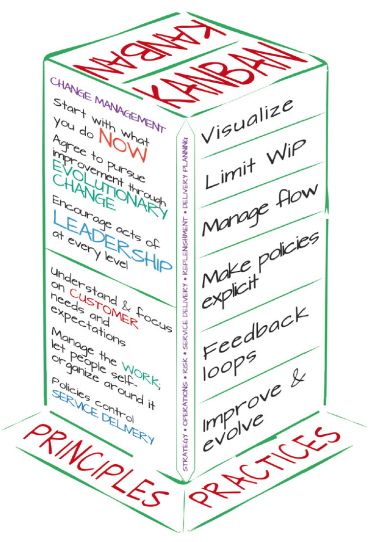


- **Service request Manager**
  - Se encarga de gestionar la demanda y los requisitos dentro del sistema Kanban, manejando las relaciones con los stakeholders y fomentando la transparencia dentro del sistema en torno a la priorización del trabajo. Alternativamente, este rol se puede llamar Product Manager, Product Owner o Service Manager.
- **Service Delivery Manager**
  - Es responsable del flujo de trabajo dentro de un sistema Kanban y/o determinados ítems de trabajo y facilita el Kanban Meeting y el Delivery Planning. Algunos nombres alternativos son Flow Manager, Delivery Manager o incluso Flow Master.

Metodologías Ágiles

- Kanban

Prácticas



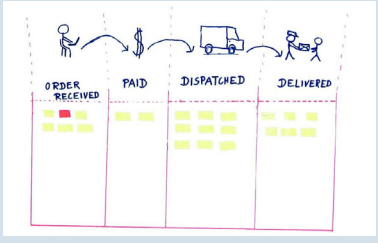
Metodologías Ágiles

- Kanban



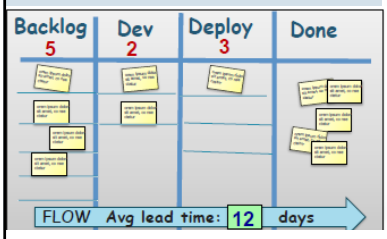
1.Visualizar

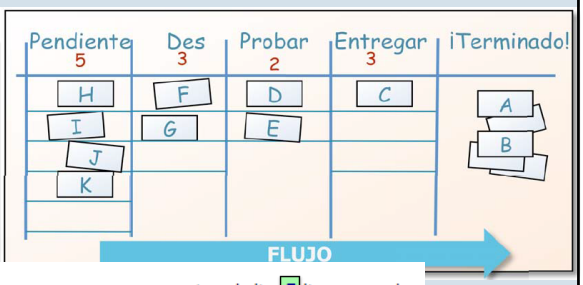
Primero, divide el flujo de trabajo desde el momento en que lo empiezas hasta que termina en pasos distintos y dibuja una columna para cada uno. Después, compra unos stickies. Anota cada tarea en una nota adhesiva aparte. Usa diferentes colores para distintos tipos de trabajo. Ponlos en la pizarra. Cada tarea se moverá de izquierda a derecha hasta que termina y sale del flujo de trabajo.

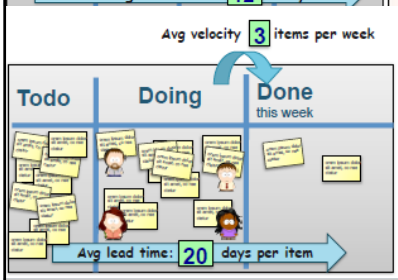


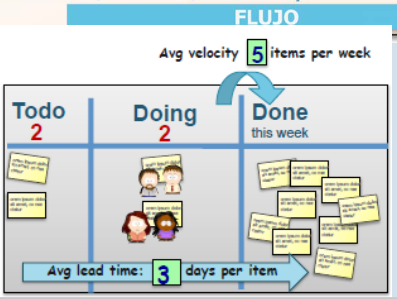
Metodologías Ágiles

- Kanban









•*Henrik Kniberg*, **Kanban intro,** 2011

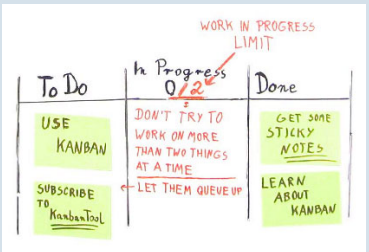
Metodologías Ágiles

- Kanban



2. Limitar el Work in Progress (WIP)

Con las dos manos solo puedes hacer malabares con unas pocas pelotas antes de empezar a soltarlas. Kanban se trata de mantener el flujo y eliminar residuos. Empieza poniendo límites a las columnas en las que se realiza el trabajo. Los Límites en Progreso buscan garantizar un flujo de trabajo alto y fluido y eliminar diferentes tipos de residuos



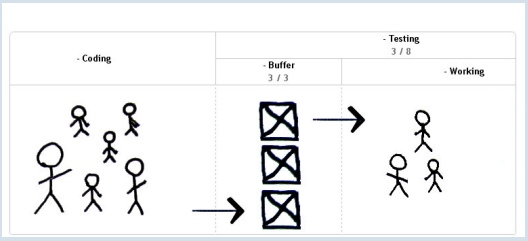
Metodologías Ágiles

- Kanban



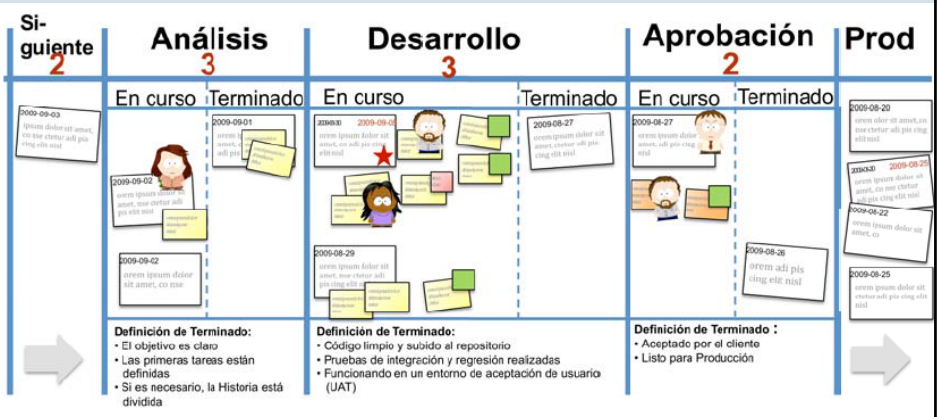
3. Controla el flujo de trabajo

Es fácil generar fricciones entre diferentes equipos, especialmente cuando uno rinde mejor y exige más trabajo del que otro puede manejar realmente. Una solución a esto es un sistema de pull, donde el siguiente equipo tira del trabajo solo cuando está preparado para ello. Puedes implementar un sistema de pull añadiendo un buffer de capacidad limitada entre equipos.



Metodologías Ágiles

- Kanban



Kanban and Scrum - Making the Most of Both, *2009*

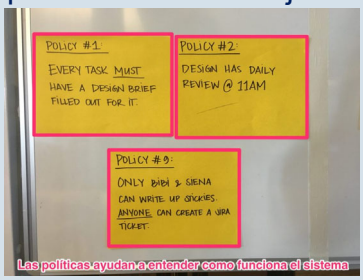
Metodologías Ágiles

- Kanban



4. Visibiliza las políticas

- Cualquier decisión tomada debe mostrarse explícitamente
  - ¿Quién se encarga de mover fichas?
  - ¿Con qué frecuencia se revisa el consejo?
- Las normas de política deben ser ajustadas por los equipos



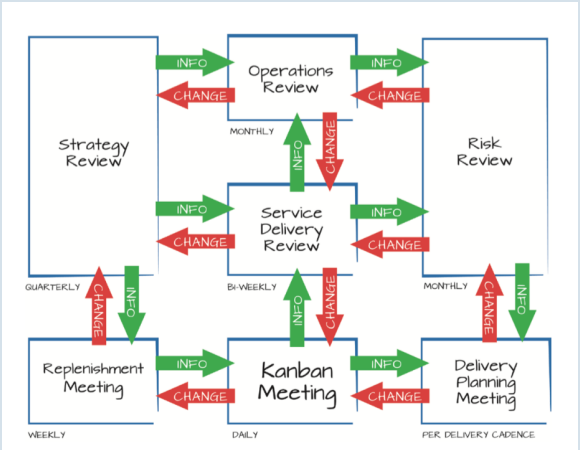
Las políticas ayudan a entender como funciona el sistema

Metodologías Ágiles

- Kanban



5. Bucles de realimentación



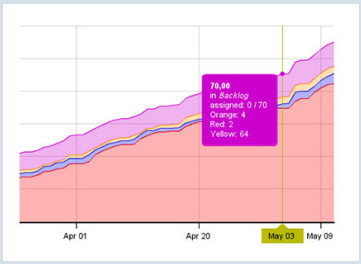
Metodologías Ágiles

- Kanban



6. Mejora y evoluciona

La mejor herramienta para medir el rendimiento de Kanban es el diagrama de flujo acumulativo. Cada día, para cada columna, marca cuántas tareas hay en ella o en algún punto más avanzado en el flujo de trabajo. Esto generará un gráfico con aspecto de montaña, que ofrece una visión del proceso, muestra el rendimiento pasado y permite predecir resultados futuros.

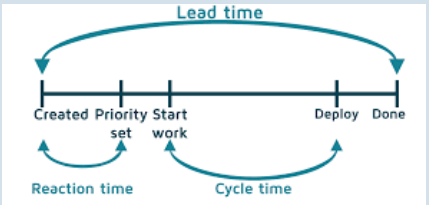


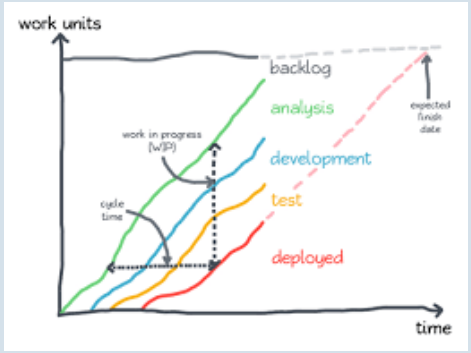
Metodologías Ágiles

- Kanban



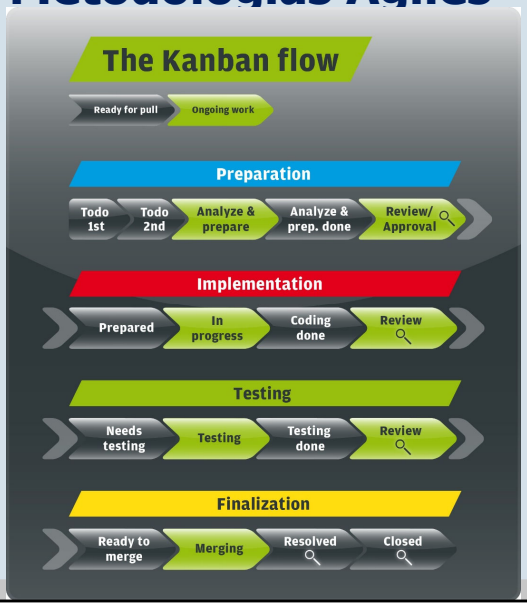
6. Mejora y evoluciona





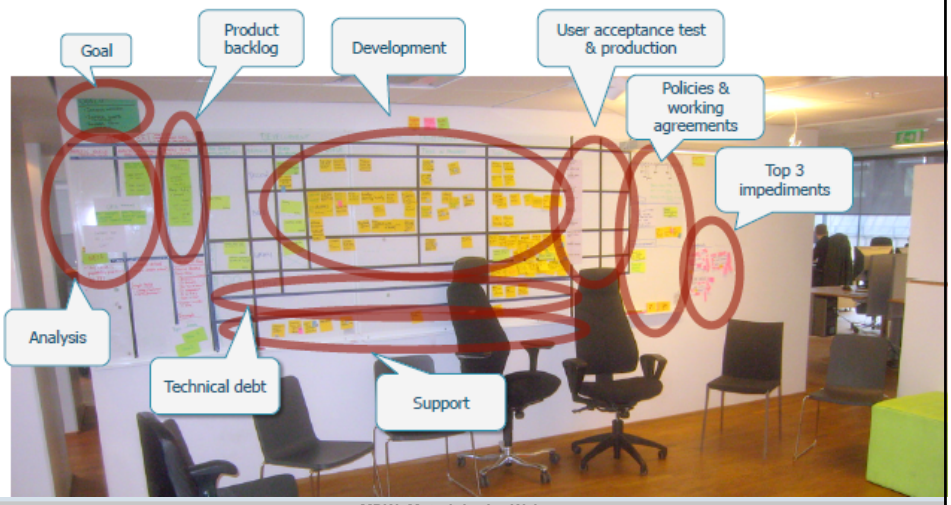
Metodologías Ágiles

- Kanban



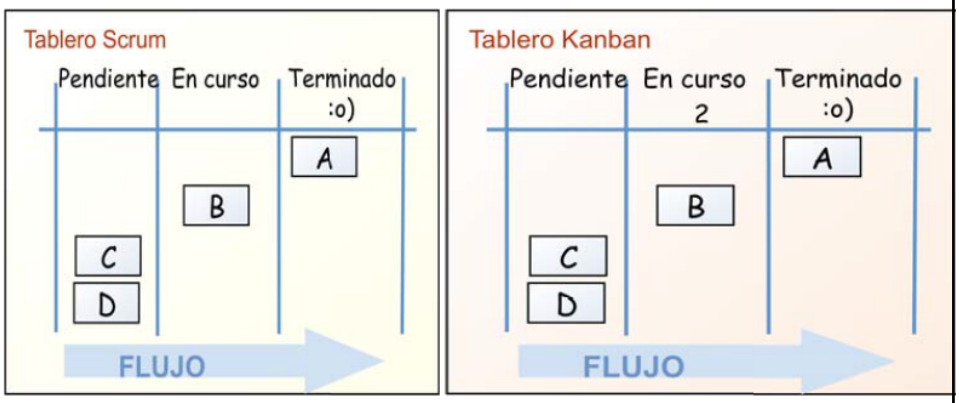
Metodologías Ágiles

- Kanban



Metodologías Ágiles

- Kanban



SCRUM



Bases

- **Flexibilidad**



Adaptabilidad



Productividad



- Método de gestión para procesos de desarrollo de sistemas

- NO propone ninguna técnica de desarrollo de software en particular

- Define cómo ha de trabajar un equipo para producir trabajo de calidad en un entorno cambiante.

Bases

Scrum en 90 palabras

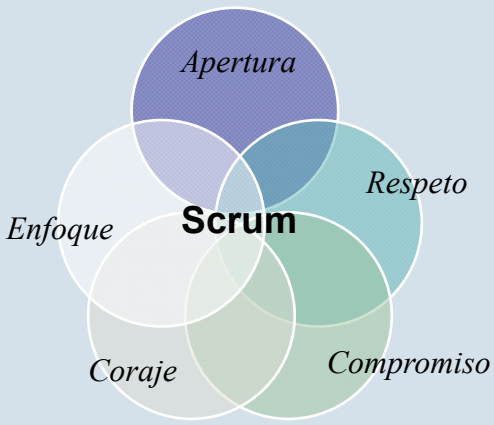
- Scrum es un proceso ágil que permite centrarnos en ofrecer el valor más alto de negocio en el menor tiempo posible.
- Nos permite rápidamente y en repetidas ocasiones inspeccionar software real de trabajo (cada dos semanas o un mes).
- El negocio fija las prioridades. Los equipos se auto-organizan a fin de determinar la mejor manera de entregar las funcionalidades de más alta prioridad.
- Cada dos semanas o un mes, cualquiera puede ver el software real funcionando y decidir si liberarlo o seguir mejorándolo en otro sprint.

Características

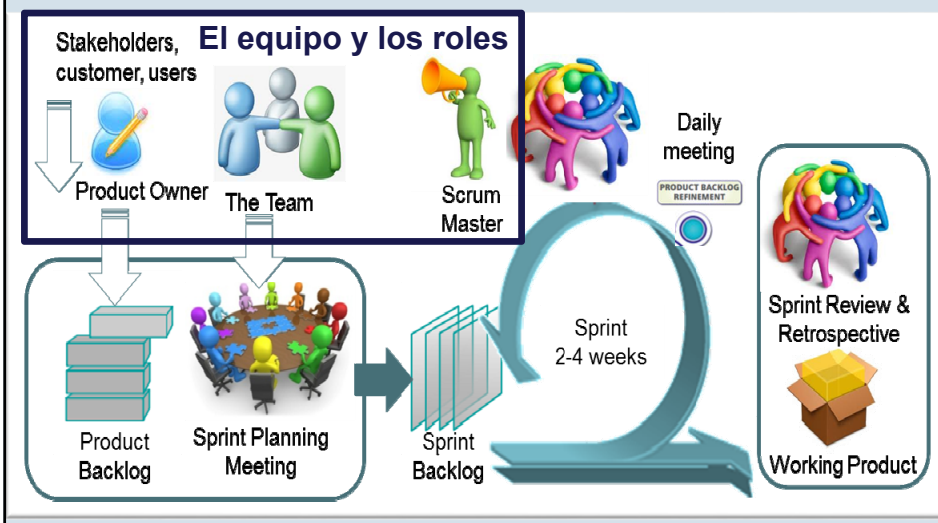
Scrum:

- Ayuda a las personas, equipos y organizaciones a **generar valor** a través de soluciones **adaptables** para problemas complejos.
- Es **simple** y deliberadamente **incompleto**, definiendo solamente las partes necesarias, en lugar de proporcionar a las personas instrucciones detalladas.
- Permite emplear diversos **procesos, técnicas y métodos**.
- Emplea un enfoque **iterativo e incremental** para optimizar la previsibilidad y controlar el riesgo.
- Involucra a grupos de personas que colectivamente tienen todas las **habilidades y experiencia** para hacer el trabajo y compartir o adquirir tales habilidades según sea necesario.

Valores Personales



Proceso Global



Equipos

Multidisciplinar
Dueño del producto
Analistas
Diseñadores
Encargados de Calidad (QA) y de la documentación
Desarrolladores



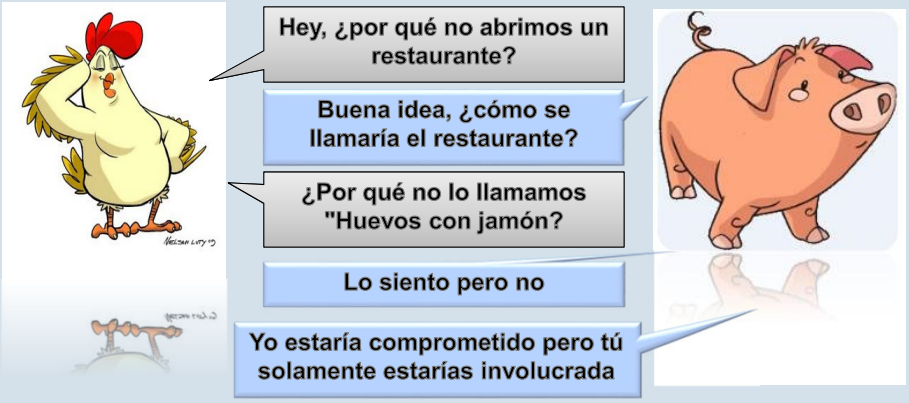
Pequeño
De 5-10 personas

Auto-organizado
Los miembros del equipo se organizan entre ellos

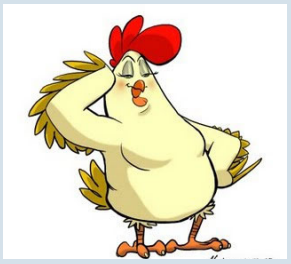
Auto-gestionado
El proyecto lo gestionan los propios miembros del equipo

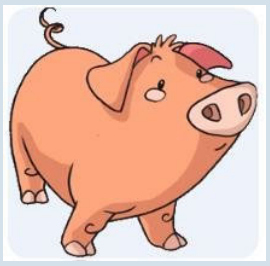
Roles *(Proyecto)*

- Un cerdo y una gallina se encuentran por la calle y.....



Roles *(Proyecto)*





- Stakeholders:
  - Usuarios Finales
  - Cliente
  - Etc.
- Gerentes

- Equipo Scrum
  - Scrum Master
  - Product Owner
  - Miembros del Equipo de Scrum (Desarrolladores)

Roles Equipos Scrum (*Scrum Team*)

Scrum prescribe 3 roles para sus equipos



Product Owner



Desarrolladores

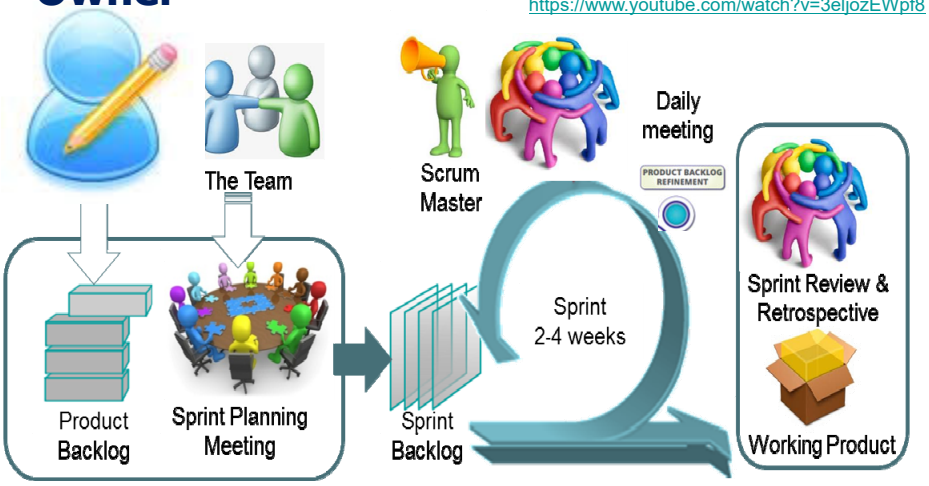


Scrum Master

Roles del *Scrum Team*

Product Owner

https://www.youtube.com/watch?v=3eljozEWpf8



Roles del *Scrum Team*

- **Responsabilidades**
  - Consolida las necesidades de los usuarios, stakeholders y otras partes interesadas para crear una visión única y priorizada de los requisitos del sistema.
    - Acepta o rechaza el resultado del trabajo del equipo.
  - Define las características del producto, decide fechas de entregables y contenido del mismo.
    - Priorizar las características de acuerdo al valor de mercado.
    - Si es necesario ajustar funcionalidades y prioridades en cada iteración.

Product Owner



Roles del *Scrum Team*

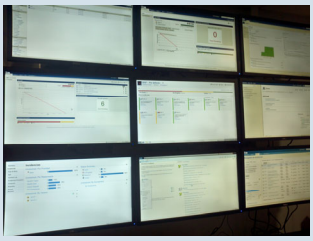
- **Responsabilidades**
  - Tener una visión del:
    - Producto
    - Entregas (*releases*)
    - Sprint
  - Respetar las estimaciones del equipo.
  - Mantener la comunicación.
  - Es responsable de:
    - Los beneficios del producto ROI (*Return On Investment*)
    - Coste

Product Owner



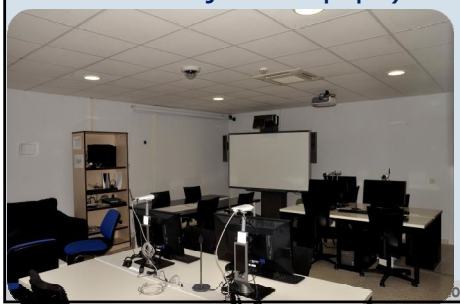
Roles del *Scrum Team*

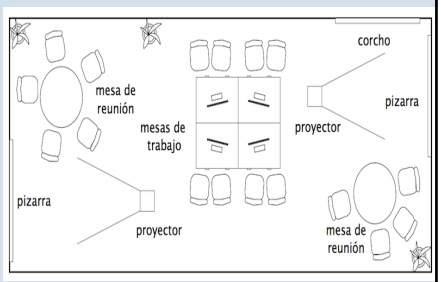
- **Lugar de trabajo**
  - **Mantén al Dueño de Producto a mano**
    - Cerca pero no en el mismo sitio (sala de trabajo del equipo)



Product Owner



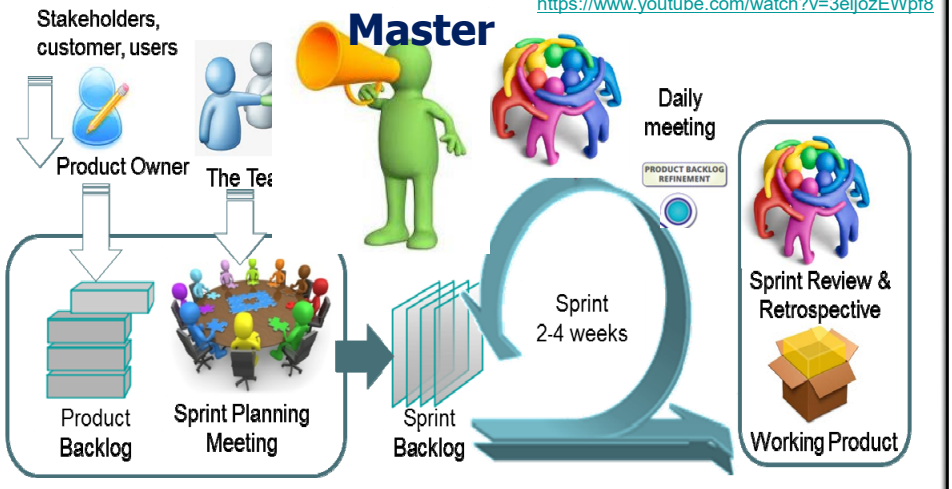




Roles del *Scrum Team*

Scrum Master

https://www.youtube.com/watch?v=3eljozEWpf8



Roles del *Scrum Team*

Scrum Master



- Representa a la gestión del proyecto.
  - Es el propietario de la implementación del método
- Es el líder del equipo.
- Aunque es incorrecto, a veces se le llama *Project Manager* pero:
  - Lo que **NO** es o **NO** hace:
    - Jefe de proyecto
    - No delega sus tareas
    - No cambia las planificaciones
    - No toma decisiones
    - No se encarga de cumplir fechas, presupuestos, …

Roles del *Scrum Team*

Scrum Master



- Responsable de promover los valores y prácticas de Scrum.
- Se asegura de que el equipo es completamente funcional y productivo.
- Escudo del equipo ante interferencias externas.
- Puede ser un rol de tiempo completo o a tiempo parcial:
  - Se pueden gestionar varios sprint al mismo tiempo.
  - Puede actuar como otro miembro del equipo.

Roles del *Scrum Team*

Scrum Master



- Encargado de fomentar un entorno donde:
  1. El Product Owner ordena el trabajo de un problema complejo en el Product Backlog.
  2. El equipo de Scrum convierte una selección del trabajo en un Incremento de valor durante un Sprint.
  3. El equipo de Scrum y sus partes interesadas (stakeholders) inspeccionan los resultados y realizan los ajustes necesarios para el próximo Sprint.
  4. *Repetir*

Roles del *Scrum Team*

Scrum Master



- ***Perfil*** del Scrum master
  - Personas que confían en el éxito del proyecto.
  - Tienen autoridad
  - Conoce perfectamente el entorno de negocio del cliente, las necesidades y el objetivo.
  - Tener atribuciones suficientes para tomar las decisiones necesarias durante el desarrollo.
  - Es recomendable conocer y haber trabajado previamente con el mismo equipo

Roles del *Scrum Team*

Desarrolladores

https://www.youtube.com/watch?v=vLqCkj0PvtE



Roles del *Scrum Team*

Desarrolladores



- **Actividades que realiza**
  - Cumplir con el sprint.
  - Realizar estimaciones.
  - Planificar su propio trabajo.
  - Tener la autoridad de realizar cualquier cosa necesaria para conseguir el objetivo.
  - Se compromete con el *Product owner* en las tareas a realizar en cada iteración, y luego las hace.
  - Al final de la iteración le muestran al *Product owner* el resultado.

Roles del *Scrum Team*

Desarrolladores



- Responsabilidades
  - Auto-organización y uso de tecnologías ágiles (grupo de trabajo vs equipo).
    - ***Grupo de Trabajo***:
      - Conjunto de personas que realizan un trabajo con una asignación específica de tareas, responsabilidades y siguiendo un proceso o pautas de ejecución.

Roles del *Scrum Team*

Desarrolladores



- ***Equipo***:
  - Grupo de personas que tiene espíritu de colaboración, y un propósito común: conseguir el mayor valor posible para la visión del cliente.
    - » Un equipo Scrum:
      - » Responde en su conjunto.
      - » Trabajan de forma cohesionada y auto-organizada.
      - » No hay un gestor que delimita, asigna y coordina las tareas. Son los propios miembros del equipo los que realizan estas funciones.

Roles del *Scrum Team*

- **Responsabilidades:**
  - Determina los resultados del trabajo.
  - Hacer todo lo necesario (dentro de los límites de las guías del proyecto) para alcanzar el objetivo de la iteración.
  - Realizar estimaciones.
  - Organizarse a si mismo y organizar su trabajo.
  - Realizar demostraciones del resultado del trabajo ante el *Product Owner* y los *stakeholders*.

Desarrolladores



Roles del *Scrum Team*

- **Funcionamiento**
  - Todos los miembros conocen comprenden la visión del propietario del producto.
  - Aportan y colaboran con el propietario del producto en el desarrollo de la pila de producto.
  - Comparten de forma conjunta el objetivo de cada sprint y la responsabilidad del logro.
  - Todos los miembros participan en las decisiones.
  - Se respetan las opiniones y aportaciones de todos
  - Todos conocen la metodología de trabajo con Scrum.

Desarrolladores



Roles del *Scrum Team*

- **Funcionamiento**
  - Desarrollo secuencial vs. superpuesto.

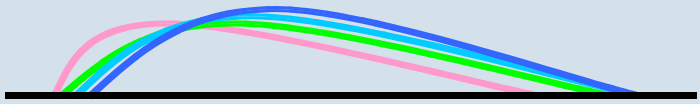
Desarrolladores



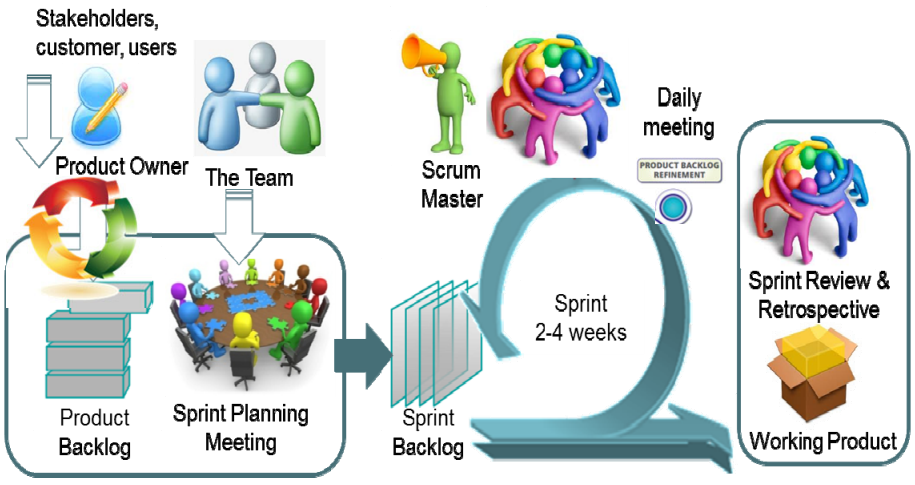
Requisitos	Diseño	Código	Test

En lugar de una cosa detrás de otra ...

...los equipos Scrum hacen un poco de todo todo el tiempo

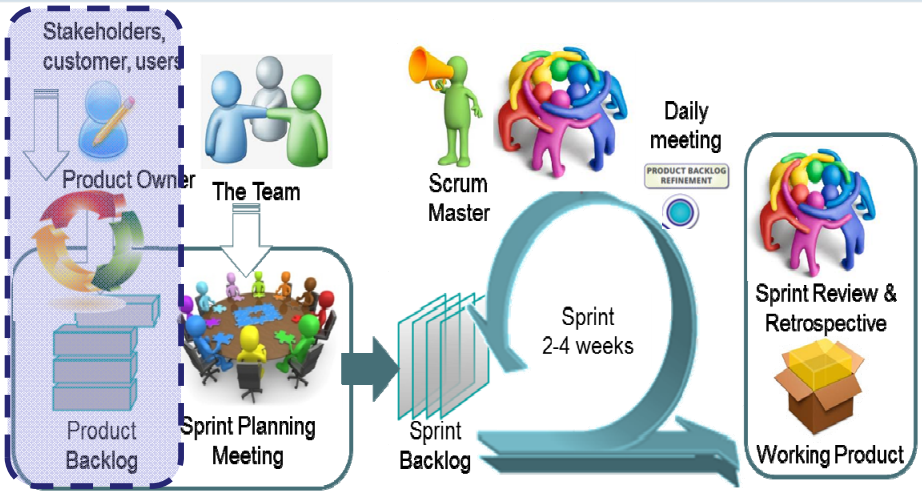


Proceso Global SCRUM



Primera Actividad Scrum

Pregame /Iteración Inicial / Workshop de Creación del Product Backlog



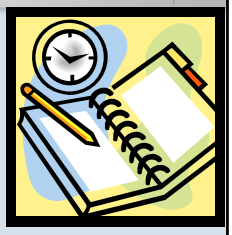
Creación del Product Backlog

- Reunión de planificación de la que se obtiene el **Objetivo del Producto** y el **Product Backlog**
- Planificación conjunta, sencillez, claridad
- Planificación del Product Backlog: Definición del Sistema Software
  - **Producto**
    - Considera **los requisitos** que debe incluir un producto software
  - ***Release***
    - Definir las distintas versiones entregables del producto hasta la entrega del producto final



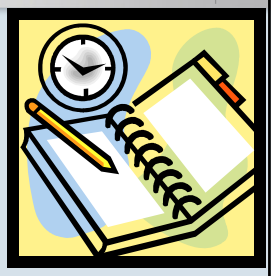
Se describen en términos de Historias de Usuario Son capturados como elementos de una lista de "Product Backlog"

Creación del Product Backlog

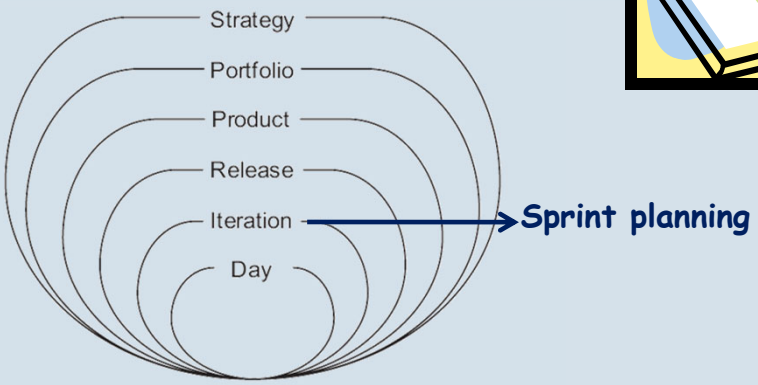


- **Planificación Ágil**
  - Establece un conjunto de expectativas base del proyecto (**alcance**).
  - Soporta el proceso de toma de decisiones.
  - Facilita la comunicación.
  - Ventajas que aporta:
    - Reduce riesgos
    - Reduce incertidumbre
    - Da un mejor soporte a la toma de decisiones
    - Establece la verdad
    - Proporciona información

Sprint Planning



- **Planificación Ágil**
  - En Scrum



The Planning Onion. Agile Estimating And Planning
Mike Cohn, , 2005

Sprint Planning



- Reunión de planificación de la iteración
  - ¿Qué se hace?
    - Selección de historias del usuario a partir del ***Product Backlog*** para:
      - Definición del ***Working Product***
      - Construcción del ***Sprint Backlog***
    - Definición del **objetivo** del Sprint
    - Definición del lugar y el momento para realizar las *daily meetings* durante la iteración
    - Fecha para hacer:
      - La reunión de revisión: Demostración del Sprint
      - La reunión retrospectiva

Sprint Planning



- ¿Quién participa?
  - Product Owner
  - Scrum Master
  - Equipo Scrum
- Planificación de la **duración** del Sprint:
  - Tanto tiempo como el Product Owner se comprometa a no introducir cambios en el sprint y dejar trabajar unas semanas sin interrupciones al equipo.



Sprint Planning



- **Resumen:** Una reunión para preparar el Sprint, normalmente dividida en dos partes ("qué" y "cómo").

- **Participantes:**
- Parte Uno: Product Owner, Team, ScrumMaster.
- Parte Dos: Equipo, ScrumMaster, Product Owner

- **Duración:**
  - Cada parte está limitada a una hora (no más de 2) por semana de Sprint.



Sprint Planning



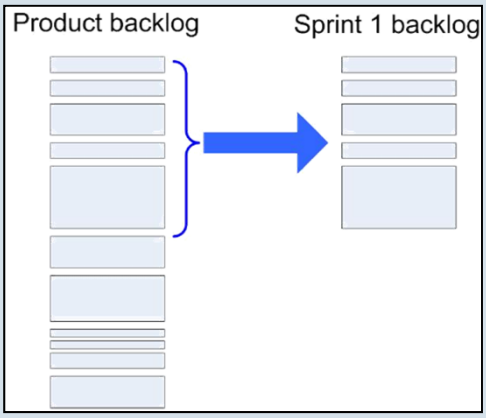
- **Planificación de Sprint Parte Uno**
  - La Parte Uno se centra en entender qué quiere el Product Owner y por qué es necesario.
  - El PO y el equipo revisan los elementos de alta prioridad en el Product Backlog que el PO está interesado en implementar este Sprint. Objetivo Sprint
  - Estos elementos habrán sido bien analizados durante el refinamiento del Product Backlog.
  - Cada artículo debe dividirse lo suficientemente pequeño como para que se estime que requiere considerablemente menos que todo el Sprint. (un cuarto o menos de un Sprint por todo el equipo)



Sprint Planning

- ¿Qué Historias de Usuario incluir en el sprint?





Sprint Planning

- ¿Qué Historias de Usuario incluir en el sprint?



Product Owner



Alcance

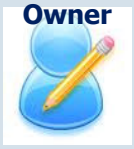
Importancia

Product Owner



Variables de una Historia de Usuario

Product Owner



Calidad
Externa e Interna

Scrum Team



Estimación

Scrum Team



Sprint Planning



- **Planificación de Sprint Parte Dos:**
  - Se centra en cómo implementar los elementos que el equipo decide abordar en la Parte Uno.
  - El equipo hace previsiones ordenadas sobre la cantidad de ítems que pueden completar al final del Sprint, comenzando por la parte superior del Product Backlog.
  - El equipo decide cuánto trabajo realizará, en lugar de que el Product Owner se lo asigne.
  - El Product Owner no tiene control sobre cuánto se compromete el equipo.



Sprint Planning



- ¿Cómo realizamos la reunión de ***Sprint Planning***?
  1. **Refinamiento del Product Backlog**
     1. Dudas alcance y calidad externa
     2. Contenido
     3. Descomposición de US
     4. Adición de US
     5. Eliminación de US
     6. Re-priorización: Actualización de ratios de importancia
     7. Re-estimación

				New Estimates at Sprint ...					
Priority	Item	Details (wiki URL)	Initial Size Estimate	1	2	3	4	5	6
1	As a buyer, I want to place a book in a shopping cart (see UI sketches on wiki page)	...	5						
2	As a buyer, I want to remove a book in a shopping cart	...	2						
...	...	...		...					
7	As a shopper, I want to create and save a wish list	...	40						
8	As a shopper, I want to to add or delete items on my wish list	...	20						

Sprint Planning



- ¿Cómo realizamos la reunión de ***Sprint Planning***?
  2. **Selección** de Historias del ***Product Backlog*** a partir de la estimación
  3. Se establece cuándo y dónde se van a hacer las **daily meetings**
  4. Se establece cuándo realizar la **revisión y retrospectiva** del sprint

Sprint Planning



- Ejemplo:

Sprint planning meeting: 13:00-17:00 (10-minute break each hour)

- **13:00-13:30** – Product owner goes through sprint goal and summarizes product backlog. Demo place, date, and time is set.
- **13:30-15:00** – Team time-estimates, and breaks down items as necessary. Product owner updates importance ratings as necessary. Items are clarified. "How to demo" is filled in for all high-importance items.
- **15:00-16:00** – Team selects stories to be included in sprint. Do velocity calculations as a reality check.
- **16:00-17:00** – Select time and place for daily scrum (if different from last sprint). Further breakdown of stories into tasks.

Sprint Planning



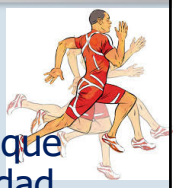
- Resultados del ***Sprint Planning***:
  1. Objetivo del sprint.
  2. Lista de los miembros del equipo (grado de compromise si no es el 100%).
  3. Sprint backlog (= Lista de historias de usurios incluidas en el sprint).
  4. Fecha y lugar de las reunions diarias.
  5. Fecha y lugar de la reunion de *review.*
  6. Fecha y lugar de la reunion de *retrospective.*

Proceso Global

Sprint



Sprint



- **Concepto:**
  - Periodo de tiempo (**timebox**) durante el que se desarrolla un incremento de funcionalidad.
  - Constituye el núcleo de Scrum.
  - Duración:
    - Típica: 2–4 semanas.
    - Máxima: 1 mes de calendario.
    - Constante: conduce a un mejor ritmo.
  - El producto es diseñado, codificado y probado durante el Sprint.
  - ***Working Product***: En cada iteración se entrega un producto al cliente.

Sprint



- **Concepto:**
- Durante el sprint:
  - La calidad no disminuye.
  - El trabajo pendiente del producto se refina según sea necesario.
  - El alcance del producto se puede clarificar y renegociar con el Propietario del Producto a medida que se aprende más.
  - Se realiza todo el trabajo necesario para alcanzar el objetivo del producto: Planificación (Sprint Planning), Daily Scrums, Revisión del Sprint (Sprint Review) y la Retrospectiva (Sprint Retrospective).
  - No se hacen cambios que pongan en peligro el Objetivo Sprint.

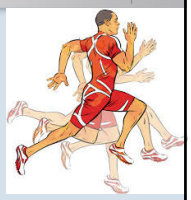
Sprint



- **Concepto:**
- Durante el sprint no se puede modificar el trabajo que se ha acordado en el Sprint Backlog
  - Sólo es posible cambiar el curso de un sprint, **abortándolo**, y **sólo** lo puede hacer el ***Scrum Master*** si decide que no es viable por alguna de las **razones** siguientes:
    - La tecnología acordada no funciona.
    - Las circunstancias del negocio han cambiado.
    - El equipo ha tenido interferencias.



Sprint



- **Objetivo:**
- Una breve declaración de la meta del sprint
- Algo que no se ha logrado hasta el momento
- Ejemplos:
  - Versión beta lista!!
  - La aplicación se ejecutará en SQL Server, además de Oracle





Sprint

- **Definición de DONE (Terminado)**
  - Descripción formal del estado del Incremento cuando cumple con las medidas de calidad requeridas para el producto.
  - Deben acordarlo el product owner y el equipo antes del primer sprint
    - Lista de criterios que un incremento (normalmente una US) debe cumplir para considerarse acabado
      - Si no se cumplen todos los criterios no suma a la velocidad conseguida durante el sprint
    - Posibles definiciones:
      - "Listo para ser implantado/desplegado en producción"
      - "Listo para ser desplegado en el servidor de test y listo para los test de aceptación"

Buenas prácticas



- **Desarrollo y Patrones**
  - Scrum no propone ninguna técnica de desarrollo software particular.
  - Recomendación: uso de patrones. Dos niveles de abstracción:
    - Patrón de arquitectura software
    - Patrón de diseño software

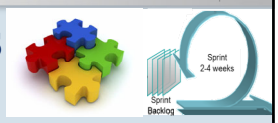




+ abstracción

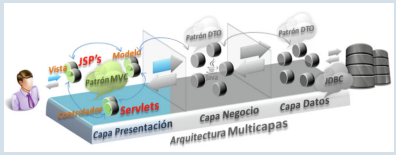
- abstracción

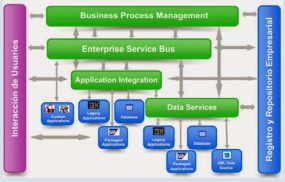
Buenas prácticas

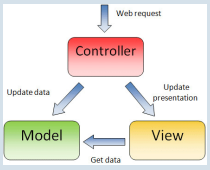


- **Desarrollo y Patrones**
  - Patrón de arquitectura software: estrategia que expresa un esquema de organización estructural esencial para un sistema sw, que consta de subsistemas, sus responsabilidades e interrelaciones.

    Ejemplos: arquitecturas multinivel (en tres capas: presentacion-negocio-datos), arquitectura dirigida por eventos, arquitectura orientada a servicios, modelo-vista-controlador,...

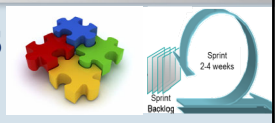






Mejora diferentes cualidades del software: aumento de rendimiento, aumento de reusabilidad (reduciendo acoplamiento),...

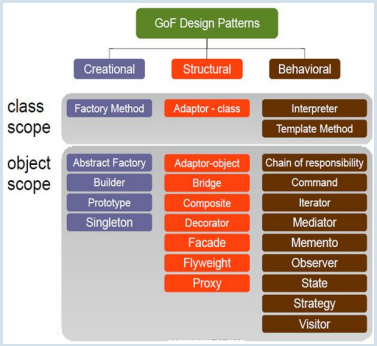
Buenas prácticas



- **Desarrollo y Patrones**
  - Patrón de diseño software

    solución general a un problema general que puede adaptarse a un problema concreto. Evitan la reiteración en la búsqueda de soluciones a problemas ya conocidos y solucionados anteriormente.

    Clasificación GoF: creacionales, estructurales y de comportamiento.



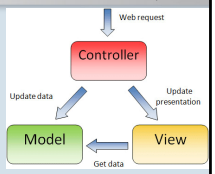
Design patterns.Elements of reusable Object Oriented Software
Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides (GoF-Gang of Four), 1994
Head First Design Patterns
Eric Freeman, Elisabeth Robson, Bert Bates, Kathy Sierra, 2004

Buenas prácticas

- **Desarrollo y Patrones**
  - **MVC (Model-View-Controller)**
    - Patrón de arquitectura software. Separa los datos y la lógica de negocio de una aplicación de la interfaz de usuario y la gestión de eventos.
    - Basado en la **reutilización** de código y **separación de conceptos** (facilita el desarrollo de sw y su posterior **mantenimiento**).
    - Tres componentes (objetos):
      - Modelo. Representación de la información.
      - Vista. Visualización de la información.
      - Controlador. Interacción del usuario (cómo reacciona el interfaz a los eventos generados por el usuario).



Buenas prácticas



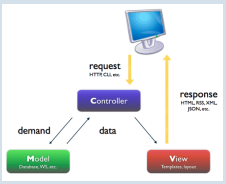
- **Desarrollo y Patrones**
  - **MVC (Model-View-Controller)**
    - **Sin MVC:**
      - Modelo, vista y controlador se diseñan juntos o
      - Modelo, vista y controlador están fuertemente acoplados .

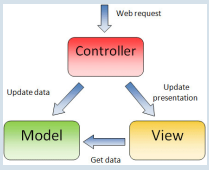
      Baja o nula reusabilidad.
    - **Con MVC:**
      - El modelo y la vista están desacoplados mediante un protocolo "subscripción/notificación".
      - Cada vez que cambian los datos del modelo, el modelo notifica estos cambios, de forma que la o las vistas que están "subscritas" a ese modelo se actualizan.
      - Facilita tener varias vistas para el mismo modelo para ofrecer distintas representaciones de la misma información.
      - El modelo no tiene "conocimiento" de las vistas, por lo que se pueden crear nuevas vistas para el mismo modelo sin necesidad de modificar el modelo en sí.

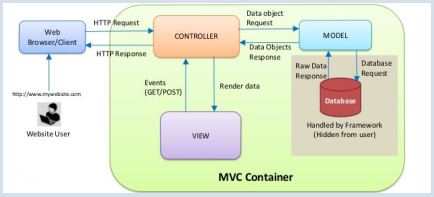
    Al estar el modelo desacoplado de sus posibles vistas se favorece la reutilización.

Buenas prácticas

- **Desarrollo y Patrones**
  - **MVC (Model-View-Controller)**
    - MVC surgió en el contexto del paradigma de la OO y en el desarrollo de aplicaciones de escritorio, pero hoy en día se aplica a frameworks para desarrollo Web. Dependiendo de la implementación concreta, la forma de avisar a las vistas del cambio del modelo varía (en algunos casos es el controlador el encargado de hacerlo).







  - MVC mejora la reutilización de código y separación de conceptos (facilita el desarrollo de sw. y su posterior mantenimiento).
  - MVC favorece el patrón de arquitectura en tres capas (presentacion-negocio-datos)

Daily Meeting

- **Reglas:**
  - Diaria, reunión de 15 minutos
  - Misma hora, mismo lugar cada día
  - No para resolución de problemas



- Todo el mundo está invitado
  - Sólo los miembros del equipo, ScrumMaster y Product Owner, pueden hablar
  - El cliente puede estar, pero no pueden participar
- Ayuda a evitar otras reuniones innecesarias

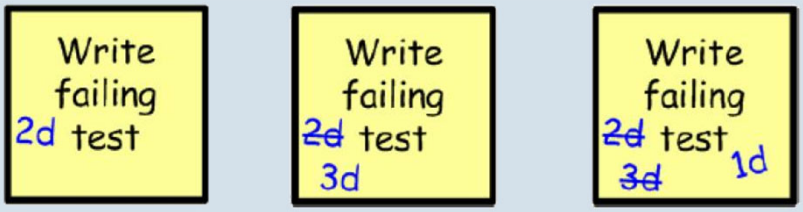
Daily Meeting

1. ¿Qué hiciste ayer?
2. ¿Qué vas a hacer hoy?
3. ¿Hay obstáculos en tu camino?

- No es dar un informe al Scrum Master
- Se trata de compromisos hacia adelante

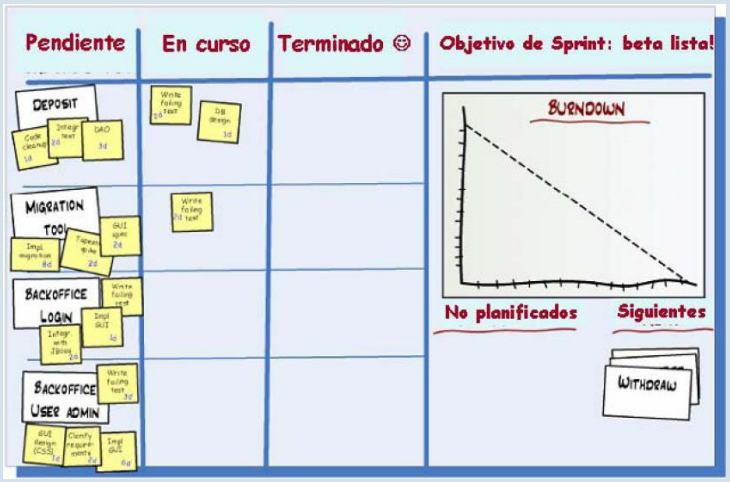
Daily Meeting

- Actualización diaria del ***Sprint Backlog/Kanban***
  - Cada persona:
    - Mueve sus tareas
    - Reestima tareas
    - Añade sus tareas no planificadas



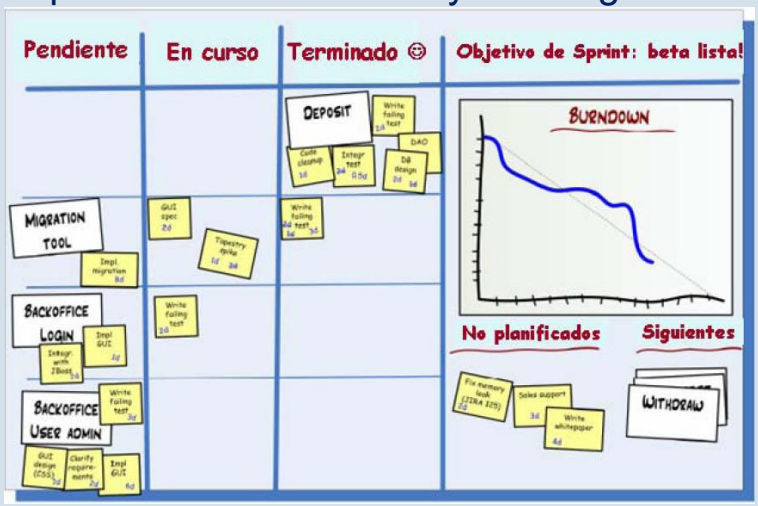
Daily Meeting

- Ejemplo de la 1ª Daily Meeting



Daily Meeting

- Ejemplo tras sucesivas Daily Meeting



Reunión Refinamiento del Backlog

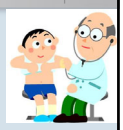
- **Resumen:** Divide los grandes elementos, analiza los elementos, reestima y vuelve a priorizar para futuros Sprints.
- **Participantes:**
  - Equipo
  - Product Owner y cualquier otro experto necesario
  - ScrumMaster
- **Duración:** Normalmente, no más del 10% de la capacidad del equipo para el Sprint.
- Frecuentemente es un taller enfocado cerca de la mitad o el final del Sprint
- Esta actividad de refinamiento no es para los elementos seleccionados para el Sprint actual; Es para objetos para el futuro.

Review Meeting



- **Sprint Demo**
  - El equipo presenta lo que se ha realizado durante el sprint.
  - Normalmente tiene la forma de presentarse una demo, o de presentar nuevas características.
  - Es informal. Reglas:
    - Tiempo de preparación 30 minutos
    - Duración no más de 2 horas.
  - Asistentes:
    - Equipo (Desarrolladores, Testers, Arquitectos del sistema,…).
    - *ScrumMaster.*
    - Propietarios del producto y clientes.
    - Cualquier toda persona interesada.

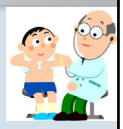
Reunión Retrospectiva



- Se realiza al finalizar cada *sprint*
- Se analiza cómo se ha trabajado:
  - Se echa un vistazo a lo que funciona y lo que no.
  - Se discute qué mejorar y se aborda cómo solucionarlo.
- Normalmente 45 minutos por semana.
- Participa todo el equipo:
  - ScrumMaster
  - Product owner
  - Equipo
  - Posiblemente clientes y otros



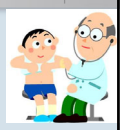
Reunión Retrospectiva



- **¿Cómo se articula?**
  - Secretario: Alguien es designado secretario.
  - El Scrum Master:
    - Muestra el ***sprint backlog.***
    - Resume con la ayuda del equipo el ***Sprint.***
  - **Ronda**: persona a persona.
  - Análisis velocidad: Estimada frente a la real.
  - Resumen de las sugerencias y mejoras.



Reunión Retrospectiva



- Formas de articular la ronda:
  - Start / Stop / Continue

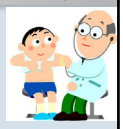


Comenzar a hacer

Dejar de hacer

Continuar haciendo

Reunión Retrospectiva



- Formas de articular la ronda:



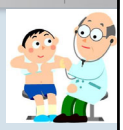


Bien

Mejorable

Mejoras

Reunión Retrospectiva



- Formas de articular la ronda:
  - +/ - / a mejorar

Se obtiene una nota de sprint

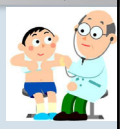


Aspectos Positivos +7

Aspectos Negativos - 5

? Aspectos a Mejorar - 2

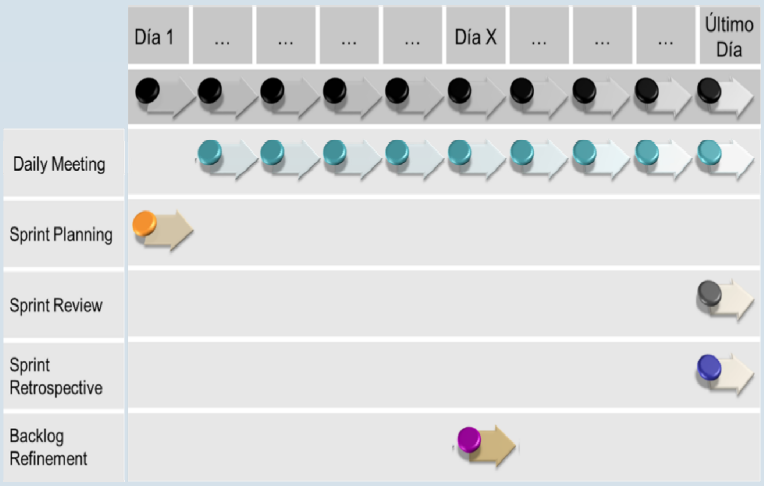
Reunión Retrospectiva



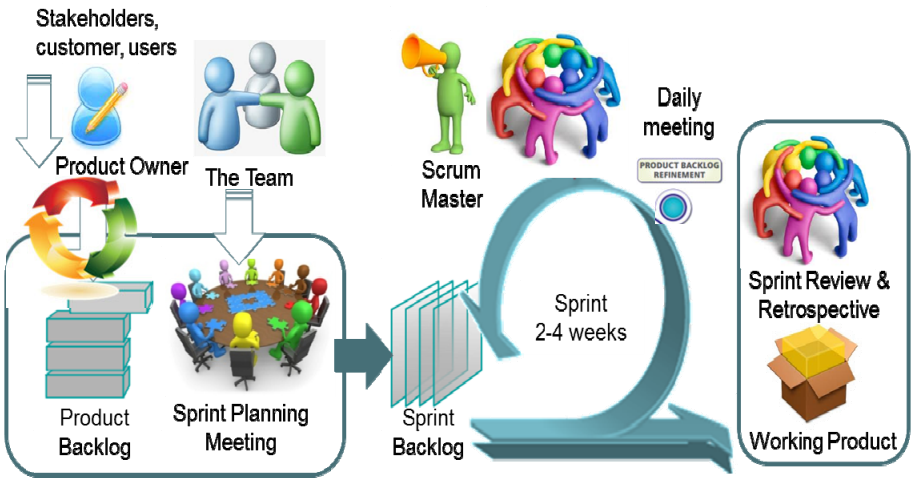
- El barco de Vela



Línea temporal de Scrum



Artefactos de SCRUM



Artefactos

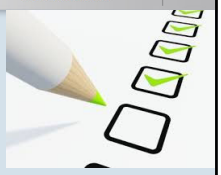


Product Backlog

- Inicialmente →Listado de Historias de Usuario sin ordenar
- Posteriormente → Listado de Historias de Usuario Ordenadas
- Es responsabilidad del **Product Owner**
  - **Definición** producto
  - **Priorización** → **Listado de Historias de Usuario ordenadas por prioridad**
  - Disponibilidad
- Es responsabilidad del **Scrum Team**
  - **Estimación**

Actividades

Artefactos



Product Backlog

- Nunca llega a ser una lista completa y definitiva
  - Es un documento dinámico que incorpora constantemente las necesidades del sistema.
  - Refinada constantemente:
    - Granularidad
    - Composición
    - Prioridad
    - Estimación
  - Se mantiene durante todo el ciclo de vida (hasta la retirada del sistema).

Artefactos



Product Backlog Ejemplo:

Backlog item	Estimación
Permitir que un invitado a hacer una reserva.	3
Como invitado, quiero cancelar una reserva.	5
Como invitado, quiero cambiar las fechas de una reserva.	3
Como un empleado de hotel, puedo ejecutar informes de los ingresos por habitación disponible	8
Mejorar el manejo de excepciones	8
...	30
...	50

+ prioridad

- prioridad

Artefactos



Product Backlog

Ejemplo:

				New Estimates at Sprint ...					
Priority	Item	Details (wiki URL)	Initial Size Estimate	1	2	3	4	5	6
1	As a buyer, I want to place a book in a shopping cart (see UI sketches on wiki page)	...	5						
2	As a buyer, I want to remove a book in a shopping cart	...	2						
...	...	...	...	...					
7	As a shopper, I want to create and save a wish list	...	40						
8	As a shopper, I want to to add or delete items on my wish list	...	20						

Prioridad

Estimación

The Scrum Primer, https://scrumprimer.org/

Artefactos

- **Sprint Backlog**
  - Compuesto por historias de usuario.
    - Selección de historias del Product Backlog:
      - Guiada: Objetivo del Sprint.
      - ¿Quién lo decide? : Siempre el equipo.
      - ¿En base a qué? → Priorización + Estimación.
  - Las historias de usuario se descomponen en tareas:
    - La descomposición:
      - La realiza el equipo.
      - No se incluye en el Product Backlog.
      - La desconoce el Dueño del Producto.

Conclusiones



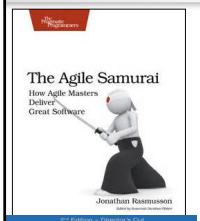
Conclusiones



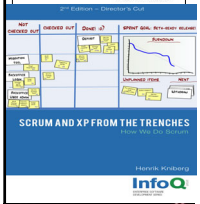
Bibliografía



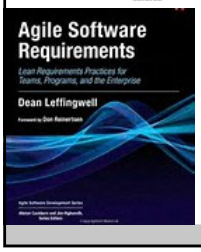
Bibliografía



- ***The Agile Samurai***, *Jonathan Rasmusson, 2010*

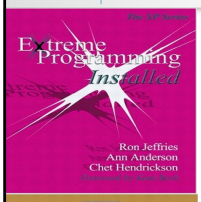


- ***SCRUM AND XP FROM THE TRENCHES***, *Henrik Kniberg, InfoQ.com, 2015.*

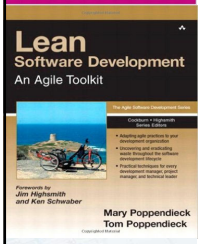


- ***Agile Software Requirements,*** *Dean Leffingwell, Addison Wesley, 2011*

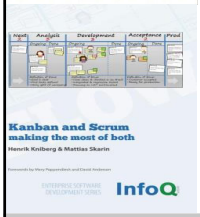
Bibliografía



- ***Extreme Programming Installed***, - Jeffries, R. , Anderson A. , Hendrickson, C. , Jeffries, R. E., *1st edition, Addison Wesley, 2000*

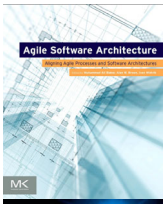


- ***Lean Software Development: An Agile Toolkit,*** *Mary Poppendieck, Tom Poppendieck, Addison Wesley, 2003*

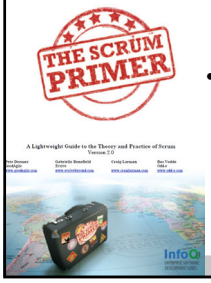


- **Kanban and Scrum - Making the Most of Both,** *Henrik Kniberg and Mattias Skarin , InfoQ.com, 2009*

Bibliografía

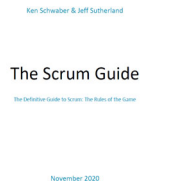






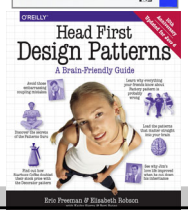
- ***Agile Software Architecture: Aligning Agile Processes and Software Architectures***, M. Ali Babar, Alan W. Brown, Kai Koskimies, and Ivan Mistrik. Ed. Morgan Kaufmann Publishers, Elsevier Inc., ISBN: 978-0-12-407772-0, 2013.
- ***Agile Estimating and Planning,*** *Mike Cohn, Prentice Hall, 2005*
- ***The Scrum Primer: A Lightweight Guide to the Theory and Practice of Scrum,*** *Version 2.0, 2012 Pete Deemer, Gabrielle Benefield, Craig Larman, Bas Vodde, 2012.*

Bibliografía



- ***The Scrum Guide. The Definitive Guide to Scrum: The Rules of the Game,*** *Ken Schwaber & Jeff Sutherland, November 2020.*





- ***Design patterns. Elements of reusable Object Oriented Software,*** *Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides (GoF-Gang of Four), 1994*
- ***Head First Design Patterns,*** *Eric Freeman, Elisabeth Robson, Bert Bates, Kathy Sierra, 2004*

Bibliografía

- ***Agile Essentials,*** ***https://www.agilealliance.org/agile-essentials/*****, 2023**
- ***Subway*** Map to Agile Practices,Agile Alliance, ***https://www.agilealliance.org/agile101/subway-map-to-agile-practices/*** **, 2020**
- ***13th Annual State of Agile Development Survey,*** *VersionOne, Inc, 2019*
- *Extreme Programming Project. http://www.extremeprogramming.org/, 2013*
- *Henrik Kniberg ,* **Kanban intro,** Leaders of Agile webinar with Kent Beck and Aslam Kahn, Oct 27, 2011
- *Sadhna Goyal,* ***Major Seminar On Feature Driven Development ,*** *Agile Techniques for Project Management and Software Engineering WS, Technical University Munich , 2008*
- **Jennifer Pérez**, Jessica Díaz, Juan Garbajosa, Agustín Yagüe: Bridging User Stories and Software Architectures: a Guidance Support for Agile Architecting. In: **Agile Software Architecture: Aligning Agile Processes and Software Architectures**. Editors: M. Ali Babar, Alan W. Brown, Kai Koskimies, and Ivan Mistrik. Ed. Morgan Kaufmann Publishers, Elsevier Inc., ISBN: 978-0-12-407772-0, pp. 215-241, 2013.

Metodologías de Desarrollo Web

Máster de Ingeniería Web
E.T.S. de Ingeniería en Sistemas Informáticos
Universidad Politécnica de Madrid
Curso 2025-2026



Jennifer Pérez y Agustín Yagüe
Universidad Politécnica de Madrid
E.T.S. de Ingeniería en Sistemas Informáticos
Departamento de Sistemas Informáticos